

Task-Driven Co-Design of Heterogeneous Multi-Robot Systems

Anonymous Authors

Abstract—Designing multi-agent robotic systems requires reasoning across tightly coupled decisions spanning heterogeneous domains, including robot design, fleet composition, and planning. Much effort has been devoted to isolated improvements in these domains, whereas system-level co-design considering trade-offs and task requirements remains underexplored. In this work, we present a formal and compositional framework for the task-driven co-design of heterogeneous multi-robot systems. Building on a monotone co-design theory, we introduce general abstractions of robots, fleets, planners, executors, and evaluators as interconnected design problems with well-defined interfaces that are agnostic to both implementations and tasks. This structure enables efficient joint optimization of robot design, fleet composition, and planning under task-specific performance constraints. A series of case studies demonstrates the capabilities of the framework. Various component models can be seamlessly incorporated, including new robot types, task profiles, and probabilistic sensing objectives, while non-obvious design alternatives are systematically uncovered with optimality guarantees. The results highlight the flexibility, scalability, and interpretability of the proposed approach, and illustrate how formal co-design enables principled reasoning about complex heterogeneous multi-robot systems.

Index Terms—Multi-robot systems, planning, scheduling and coordination, optimization and optimal control, hardware-software co-design

I. INTRODUCTION

MULTI-AGENT robotic systems are increasingly deployed in applications where the workload is spatially distributed, time-critical, or inherently parallel, including warehouse logistics [1], [2], domestic service [3], and safety-critical inspection and search tasks such as gas-leak detection [4] and de-mining [3]. In these settings, the system designer must decide not only *how to plan*, but also *what to build and deploy*: the number of robots, their types (e.g., ground vs. aerial), and the sensing/actuation/energy modules that determine each robot’s capabilities. Crucially, these choices are tightly coupled. Fleet composition shapes what planning algorithms are feasible and effective; robot-level design choices (battery sizing, actuation, sensors) determine execution horizons and sensing performance; and the task profile and environment determine which couplings matter. As a result, system design is rarely reducible to optimizing a single component in isolation [5]–[7].

A recurring challenge is that the relevant design space is both *heterogeneous* and *combinatorial*: discrete module choices and planner selections interact with continuous quantities such as time, energy, and performance. This makes exhaustive evaluation by simulation or prototyping impractical, and it makes it difficult to answer even basic questions such as: *When does a heterogeneous fleet outperform a homogeneous one? When is it worth switching planning software rather than*

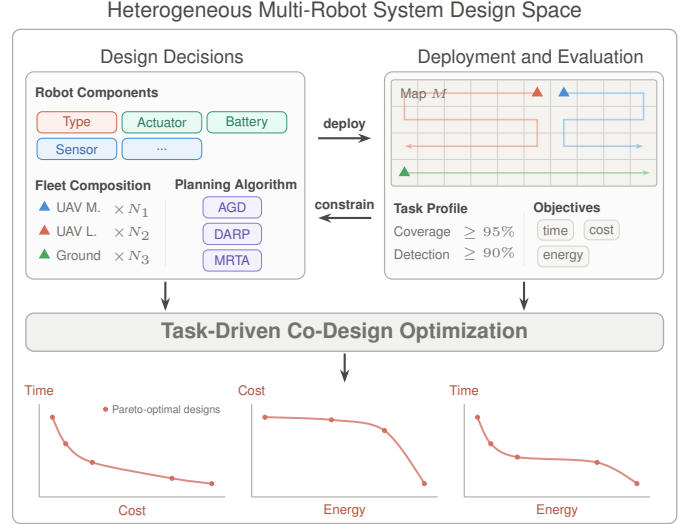


Fig. 1. Overview of the proposed task-driven co-design approach for heterogeneous multi-robot systems. Design decisions spanning robot components, fleet composition, and planning algorithm selection are jointly optimized through the monotone co-design framework subject to deployment constraints including map geometry, task profiles, and performance objectives.

upgrading hardware? How do optimal designs change as task requirements or environments change?

A. Robotics Phase Diagrams

Motivated by these questions, we advocate a “phase-diagram” perspective on multi-agent system design. In physics, phase diagrams partition external conditions into regimes with qualitatively different optimal structures (e.g., ice/water/vapor). Analogously, we would like to partition the space of *task and environment parameters* into regimes where qualitatively different *system architectures* are Pareto-optimal. For instance, boundaries where it becomes optimal to increase fleet size, introduce a new robot type, or switch to a different planning strategy. Constructing such *multi-agent robotics phase diagrams* requires a representation in which fleet design, planning, execution, and task evaluation can be varied systematically while preserving interpretability and computational tractability. This paper is a first step toward that goal.

B. Approach

We address the above challenges with a formal and compositional co-design framework based on the monotone theory of co-design [6], [8]. We introduce implementation-agnostic abstractions of (i) robots and fleets, (ii) planners that generate waypoints, (iii) executors that produce dynamically feasible trajectories under time/energy budgets, and (iv) evaluators

that score trajectories through task-relevant metrics. Each abstraction is translated into a monotone *design problem* with explicit interfaces: **functionalities** (what the component provides) and **resources** (what it requires). Interconnecting these design problems yields a system-level co-design problem that can be queried to compute task-feasible, Pareto-optimal multi-agent system designs. Importantly, because the interfaces are modular, new robot catalogs, planners, and task metrics can be integrated without changing the surrounding structure. Repeated queries over families of tasks and maps then naturally support the construction of phase-diagram-like design maps.

C. Statement of Contribution

The main contributions of this paper are as follows. First, we provide a unified formalization of heterogeneous multi-robot system design across robot design, fleet composition, planning, execution, and evaluation. We define a common interface that separates *waypoint generation* (planning), *trajectory realization under feasibility constraints* (execution), and *task scoring* (evaluation), while explicitly accommodating heterogeneous robot capabilities. Second, we derive a task-driven, system-level co-design model with formal guarantees. Building on monotone co-design, we cast robot design, fleet composition, planning, and execution as interconnected monotone design problems. This yields a principled mechanism to compute Pareto-optimal designs that satisfy a specified task profile while minimizing resources such as cost and energy. Finally, we showcase an instantiation of the models on realistic search-and-coverage missions. We demonstrate the full pipeline on coverage and probabilistic target-detection tasks using a modular robot catalog (ground and aerial platforms) and multiple planners from the literature. The case studies illustrate how optimal fleet composition and planner choice vary with task profiles and environments, providing concrete examples of the proposed phase-diagram viewpoint.

D. Paper Organization

The rest of the paper is organized as follows. We review related work in Section II and mathematical preliminaries in Section III. Section IV introduces the formal abstractions, which are translated into a co-design model in Section V. Sections VI and VII present case studies in search and coverage.

II. RELATED WORK

This paper sits at the intersection of multi-robot planning, heterogeneous fleet design, and computational co-design. We briefly review the most relevant threads and clarify the gap addressed by our framework.

A. Multi-Agent Robotic Systems and Coverage/Search Planning

As single-robot autonomy has matured, a substantial body of work has focused on multi-agent robotic systems across application domains such as environmental monitoring (including

forest-fire monitoring) [9]–[11], logistics and warehouse coordination [1], and safety-critical operations such as industrial inspection [12], disaster response [13], and search-and-rescue (SAR) [14]–[16]. Coverage and search tasks are a particularly prominent benchmark family, with surveys highlighting the breadth of coverage path planning (CPP) methods and their assumptions [3], [17]. Many CPP and search planners are developed under the assumption of a fixed fleet with given sensing and motion capabilities, often homogeneous.

Representative approaches illustrate both progress and limitations. The authors of [18] propose a heterogeneous multi-robot coverage framework that combines environment decomposition with multi-robot task allocation (MRTA) and ergodic trajectory optimization. Authors of [19] introduce DARP for equitable partitioning and coverage with homogeneous teams. Authors of [16] develop AGD for heterogeneous UAV coverage in maritime SAR by adapting spatial discretization to sensing footprints. More broadly, heterogeneous teams have been studied for their potential advantages in robustness, flexibility, and efficiency in SAR and cooperative search [15], [20]–[22]. However, these works typically treat robot capabilities and fleet composition as given inputs, rather than decision variables to be optimized jointly with the planning stack.

B. Hardware-Aware Planning and Robot-Level Co-Design

A smaller but growing literature recognizes that hardware choices and planning performance are coupled. For example, planners for guaranteed coverage under severe sensing limitations have been studied in [23], and hardware-aware viewpoints for designing minimal robots for coverage are explored in [24]. From the co-design perspective, authors of [25] formulate the selection of sensing, actuation, and computing modules as a combinatorial optimization problem subject to resource constraints. These approaches provide valuable ingredients, but they typically focus on either (i) optimizing the design of an individual platform for a fixed task model, or (ii) optimizing planning/task allocation for a fixed platform and fleet. What remains comparatively underdeveloped is an end-to-end formulation that treats *robot design, fleet composition, planner choice, and task specification* as coupled design decisions within one queryable model.

C. Monotone Computational Co-Design

Our work builds on the monotone theory of co-design [6], [8], [26], which models complex systems as interconnected design problems with partially ordered **functionalities** and **resources**. A key advantage of this paradigm is that modular subsystems can be composed while preserving formal guarantees and enabling multi-objective Pareto analysis. Monotone co-design has been applied to a range of domains, including co-design of autonomous systems that jointly reasons about hardware and autonomy stacks [27], strategic co-design in Formula 1 [28], and mobility-system design that couples vehicle characteristics with fleet sizing and infrastructure decisions [29]. These applications primarily address single-agent systems or fleets with limited heterogeneity, leaving the co-design of fully heterogeneous multi-agent systems largely

unexplored. Complementary lines of work have also begun to target co-design at the swarm/fleet level with scalability in mind [30]. These successes motivate its use for multi-agent robotics.

D. Gap and Positioning

Existing co-design applications in robotics primarily emphasize *single-platform* design or infrastructure-scale systems, whereas multi-agent deployments introduce additional decision layers: fleet size and composition, coordination software, and task-driven evaluation across environments. Moreover, capturing these layers in a way that supports systematic variation, e.g., sweeping task requirements or environment classes, is essential for the kind of design-space characterization advocated in this paper. Our contribution is therefore not a new coverage planner per se, but a *formal and compositional framework* that integrates heterogeneous robot and fleet design with planning, execution, and task evaluation. This integration provides the computational substrate needed to move from isolated design studies to phase-diagram-like characterizations of when different multi-robot system architectures become optimal as tasks and environments change.

III. MATHEMATICAL PRELIMINARIES

A. Sets and Functions

We write $f: A \rightarrow B$ for functions between sets A and B and indicate the action of f on elements by $m_A \mapsto f(m_A)$. We call A the *domain* of f , and B its *co-domain*. We will often use the broad term *map* to refer to functions. For a map $f: A \rightarrow B$, we denote the pre-image of $Y_B \subseteq B$ as the set of elements in A whose image lies in Y_B , $f^{-1}(Y_B) \stackrel{\text{def}}{=} \{m_A \in A \mid f(m_A) \in Y_B\}$. Given maps $f: A \rightarrow B$ and $g: B \rightarrow C$, their *composite* is the map $g \circ f: A \rightarrow C$ that sends $m_A \mapsto g(f(m_A))$. To express composition diagrammatically, we write: $A \xrightarrow{f} B \xrightarrow{g} C$.

We write $A \times B$ for the *Cartesian product* of sets. Its elements are tuples (m_A, m_B) , where $m_A \in A$ and $m_B \in B$. Given maps $f: A \rightarrow B$ and $g: A' \rightarrow B'$, their *product* is

$$f \times g: A \times A' \rightarrow B \times B', \\ (m_A, m'_A) \mapsto (f(m_A), g(m'_A)).$$

Given $h: A \times B \rightarrow C$, we denote its *partial evaluation* by

$$h(-, m_B): A \rightarrow C, \\ m_A \mapsto h(m_A, m_B).$$

B. Background on Orders

Definition 1 (Poset). A *partially ordered set* (poset) is a tuple $\mathcal{P} = (P, \preceq_{\mathcal{P}})$, where P is a set and $\preceq_{\mathcal{P}}$ is a partial order (a reflexive, transitive, and antisymmetric relation). If clear from context, we use P for a poset, and $\preceq$ for its order.

Definition 2 (Opposite poset). The *opposite* of a poset $\mathcal{P} = (P, \preceq_{\mathcal{P}})$ is the poset $\mathcal{P}^{\text{op}} \stackrel{\text{def}}{=} (P, \preceq_{\mathcal{P}}^{\text{op}})$ with the same elements and reversed ordering: $x_P \preceq_{\mathcal{P}}^{\text{op}} y_P \Leftrightarrow y_P \preceq_{\mathcal{P}} x_P$.

Definition 3 (Product poset). Given posets $(P, \preceq_P)$ and $(Q, \preceq_Q)$, their *product* $(P \times Q, \preceq_{P \times Q})$ is the poset with

$$(x_P, x_Q) \preceq_{P \times Q} (y_P, y_Q) \Leftrightarrow (x_P \preceq_P y_P) \wedge (x_Q \preceq_Q y_Q).$$

Definition 4 (Upper closure). Let P be a poset. The *upper closure* of a subset $X_P \subseteq P$ contains all elements of P that are greater or equal to some $y_P \in X_P$:

$$\uparrow X_P \stackrel{\text{def}}{=} \{x_P \in P \mid \exists y_P \in X_P : y_P \preceq_P x_P\}.$$

Definition 5 (Upper set). A subset $X_P \subseteq P$ of a poset is called an *upper set* if it is upwards closed: $\uparrow X_P = X_P$. We write $\mathcal{U}(P)$ for the set of upper sets of P . We regard $\mathcal{U}(P)$ as partially ordered under $U \preceq U' \Leftrightarrow U \supseteq U'$.

Definition 6 (Monotone map). A map $f: P \rightarrow Q$ between posets $(P, \preceq_P)$ and $(Q, \preceq_Q)$ is *monotone* if $x \preceq_P y$ implies $f(x) \preceq_Q f(y)$. Monotonicity is preserved by composition and products.

IV. PROBLEM FORMULATION

This section introduces the terminology and abstractions used to model and design heterogeneous multi-robot systems. We consider a modular pipeline in which (i) a *planner* assigns waypoints, (ii) an *executor* produces dynamically feasible trajectories consistent with robot capabilities and operating-time budgets, and (iii) an *evaluator* computes task-relevant performance metrics from the resulting trajectories. A *task profile* specifies which performance metric levels are required.

Pipeline abstraction: Given a fleet $\mathcal{F}$ operating in a map M , the overall pipeline can be summarized as

$$(\mathcal{F}, M) \xrightarrow{\text{planner}} \mathbf{W} \xrightarrow{\text{executor}(\cdot, \boldsymbol{\tau})} \mathbf{J} \xrightarrow{\text{evaluator}} \mathbf{m} \xrightarrow{\text{task}} \{0, 1\},$$

where $\mathbf{W}$ is a fleet-level waypoint assignment, $\boldsymbol{\tau}$ is a vector of per-robot operating-time budgets, $\mathbf{J}$ is the resulting collection of trajectories, $\mathbf{m}$ is the vector of performance metric values, and task indicates whether the task profile is satisfied.

A. Problem Overview

We begin by formalizing the *environment* in which the fleet operates.

Definition 7 (Map). A *map* M is the space of interest in which the fleet operates. We model it as a subset of the Euclidean space $M \subset \mathbb{R}^k$, where typically $k \in \{2, 3\}$ depending on whether the workspace is planar or volumetric.

Next, we formalize the abstraction used to represent individual robots.

Definition 8 (Robot). A *robot* R is a tuple $R = (\mathcal{D}, \mathcal{S})$, where $\mathcal{D}$ denotes the robot's dynamical properties (i.e., motion-related capabilities) and $\mathcal{S}$ denotes its sensing capabilities. Moreover, given a resource poset $(\mathcal{C}, \preceq_{\mathcal{C}})$, there exists a map $\text{require}: \mathcal{R} \rightarrow \mathcal{C}$ from the set of robot designs to required resources, where $\text{require}(R)$ quantifies the resources needed to realize R (e.g., cost, power, energy capacity).

The exact definition of the underlying objects that constitute a robot will be given in Section IV-B. Intuitively, a robot

exposes *capabilities* $(\mathcal{D}, \mathcal{S})$ to planning/execution, while requiring corresponding resources $\text{require}(R)$ to be realized.

Definition 9 (Fleet). A *fleet* $\mathcal{F}$ of size n is an indexed collection of robots $\mathcal{F} = \{R_1, \dots, R_n\}$.

Using an indexed collection (rather than a set) allows fleets to include multiple identical robots and avoids ambiguity about multiplicities. Throughout the paper, we allow heterogeneity: robots may differ in dynamics, sensing, and underlying state/control representations, but they interact through the abstract interfaces defined below.

1) *Planner*: The planner assigns waypoint sequences to robots so that, collectively, the fleet can attempt to accomplish the task on map M .

Definition 10 (Planner). A *planner* is a map planner $: \mathcal{F} \times M \rightarrow \mathbf{W}$, where $\mathbf{W}$ is a fleet-level collection of waypoint sequences (formalized in Definition 21).

We use the term *task* to denote the abstract mission objective (e.g., area coverage, search-and-rescue, leakage detection). In our formulation, the task influences the planner through its waypoint-generation logic, while the formal specification of task requirements is captured downstream by performance metrics and task profiles (cf. Definitions 14 and 16). This abstraction allows the framework to support a broad range of tasks without changing its overall structure; only the planner implementation changes.

Assumption 1 (Centralized offline planning). Planning is performed centrally and offline. The framework does not explicitly model inter-robot communication or distributed information exchange. Accordingly, the planner has global access to the fleet's effective dynamical and sensing capabilities and the map when computing waypoint assignments.

2) *Executor*: The *executor* converts waypoints into dynamically feasible robot trajectories, subject to per-robot operating-time budgets.

Definition 11 (Trajectory). A *trajectory* J is a map $J: [0, T_i] \rightarrow M$, which associates to each time $t \in [0, T_i]$ a position $J(t) \in M$ in the environment.

Definition 12 (Collection of trajectories). A *collection of trajectories* for a fleet $\mathcal{F} = (R_1, \dots, R_n)$ is the tuple $\mathbf{J} = (J_1, \dots, J_n)$, where J_i is the trajectory executed by robot R_i .

For each robot R_i , let τ_i denote the maximum allowed operating duration. We aggregate these budgets for all robots into $\boldsymbol{\tau} = (\tau_1, \dots, \tau_n) \in \mathbb{R}_{\geq 0}^n$.

Definition 13 (Executor). An *executor* is a map

$$\text{executor} : \mathcal{F} \times \mathbf{W} \times \mathbb{R}_{\geq 0}^n \rightarrow \mathbf{J}$$

that takes a fleet $\mathcal{F}$, waypoint collection $\mathbf{W}$, and time budget vector $\boldsymbol{\tau}$, and returns a collection of dynamically feasible trajectories $\mathbf{J} = (J_1, \dots, J_n)$ such that each J_i is defined on $[0, T_i]$ with $T_i \leq \tau_i$ for all $i = 1, \dots, n$.

Remark 1 (Time budget monotonicity intuition). The constraint $T_i \leq \tau_i$ enforces that each robot's trajectory fits

within its available operating-time budget (e.g., due to battery capacity). Providing more operating time cannot invalidate previously feasible behaviors: any trajectory feasible under budget τ_i remains feasible under any larger budget $\tau'_i \geq \tau_i$, for instance by executing the same motion and then idling.

3) *Evaluator, Metrics, and Task Profiles*: The evaluator computes performance metrics from the executed trajectories. Performance metrics may incorporate additional (possibly hidden) environment information (e.g. target locations) as internal parameters, even if that information is not provided to the planner.

Definition 14 (Performance metric). A *performance metric* is a monotone map

$$m : \mathcal{F} \times \mathbf{J} \times M \rightarrow \mathcal{P},$$

where $(\mathcal{P}, \preceq_{\mathcal{P}})$ is a poset representing the performance space. Without loss of generality, metrics are defined so that “better” performance corresponds to larger elements under $\preceq_{\mathcal{P}}$.

Remark 2. Typical examples include covered area or coverage percentage (with $\mathcal{P} = \mathbb{R}_{\geq 0}$ ordered by $\geq$) and number of detected targets (with $\mathcal{P} = \mathbb{N}$ ordered by $\geq$).

The evaluator can represent the effects of time-varying sensing phenomena, including occlusions and clutter, within the performance assessment. Given q performance metrics $m_1, \dots, m_q$ (with codomains $\mathcal{P}_1, \dots, \mathcal{P}_q$), the evaluator returns their values as a vector (performance vector).

Definition 15 (Evaluator). The *evaluator* is the map

$$\text{evaluator}(\mathcal{F}, \mathbf{J}, M) := (m_1(\mathcal{F}, \mathbf{J}, M), \dots, m_q(\mathcal{F}, \mathbf{J}, M)).$$

A task profile specifies required performance levels on the performance metric vector returned by the evaluator.

Definition 16 (Task profile). Let $\mathbf{m} = (m_1, \dots, m_q) \in \mathcal{P}_1 \times \dots \times \mathcal{P}_q$ denote the performance metric-value vector produced by the evaluator, i.e. $\mathbf{m} = \text{evaluator}(\mathcal{F}, \mathbf{J}, M)$. Let $\boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_q)$ be a vector of thresholds with $\lambda_i \in \mathcal{P}_i$. A *task profile* is the Boolean map $\text{task} : \mathcal{P}_1 \times \dots \times \mathcal{P}_q \rightarrow \{0, 1\}$ given by

$$\text{task}(\mathbf{m}) = \begin{cases} 1, & \text{if } m_i \succeq_{\mathcal{P}_i} \lambda_i \quad \forall i = 1, \dots, q, \\ 0, & \text{otherwise.} \end{cases}$$

We say the task profile is *satisfied or fulfilled* if $\text{task}(\mathbf{m}) = 1$.

Remark 3. For example, consider a coverage task with two performance metrics: $m_1 \in (\mathbb{R}, \geq)$ is the covered percentage of M and $m_2 \in (\mathbb{N}, \geq)$ is the number of detected targets. A threshold $\boldsymbol{\lambda} = (95, 2)$ requires at least 95% coverage and at least two detections.

B. Abstraction Details

In Section IV-A, we provided an overview of the design problem and its main components. In this section, we expand on the details of each component.

1) *Robot Modeling*: We now expand the definitions of $\mathcal{D}$, $\mathcal{S}$, and the resource space.

Definition 17 (Robot’s dynamical properties). A robot’s *dynamical properties* are described by a tuple

$$\mathcal{D} = (d_1, d_2, \dots, d_w),$$

where each component $d_i \in D_i$ is a dynamical parameter, and each $(D_i, \preceq_{D_i})$ is a poset. The partial orders are chosen so that $d_i \preceq_{D_i} d'_i$ means “ d'_i is at least as capable as d_i ” along that dimension.

Examples include bounds such as maximum speed $v_{\max}$, maximum acceleration $a_{\max}$, or minimum turning radius r_{turn} . These parameters may enter a robot model as constraints on admissible motion. Concretely, let the robot’s motion be governed by a parameterized model

$$\dot{x}(t) = F(x(t), u(t); \mathcal{D}), \quad (1)$$

where x is the state, u is the control input, and $\mathcal{D}$ parametrizes the admissible state/input evolution (e.g., via $|v(t)| \leq v_{\max}$, $|a(t)| \leq a_{\max}$, or curvature bounds induced by r_{turn}). Thus, $\mathcal{D}$ determines the set of dynamically feasible motions under Equation (1).

Remark 4 (Robot heterogeneity). To accommodate robots with distinct dynamical models (e.g., ground vehicles vs. UAVs), we include a categorical component $d_{\text{type}} \in D_{\text{type}}$ inside $\mathcal{D}$ that selects the appropriate model class in Equation (1). Type-specific parameters can be incorporated by extending $\mathcal{D}$ with additional coordinates (e.g., maximum thrust or climb rate for UAVs); parameters that are not meaningful for a given type can be assigned distinguished “null” values and ignored by the corresponding model F .

Similarly, we now define the sensing capabilities of a robot.

Definition 18 (Robot’s sensing capabilities). A robot’s *sensing capabilities* are described by a tuple $\mathcal{S} = (s_1, \dots, s_v)$, where each component $s_j \in S_j$ encodes a sensing-related parameter, and each $(S_j, \preceq_{S_j})$ is a poset. As for dynamics, the partial orders are chosen so that $s_j \preceq_{S_j} s'_j$ means “ s'_j is at least as capable as s_j ” along that sensing dimension.

Examples include sensing range, resolution, field-of-view, and detection certainty. To represent different sensing modalities and sensor suites, one may include a categorical or set-valued component inside $\mathcal{S}$ (e.g., modeling the sensor suite as a subset of available modalities ordered by set inclusion, so that $\text{camera} \preceq \text{camera} + \text{LiDAR}$).

Finally, the resource map $\text{require}(\cdot)$ associates to each robot design the resources required to realize its capabilities.

Definition 19 (Robot resource space). Let $(\mathcal{C}_i, \preceq_{\mathcal{C}_i})$, for $i = 1, \dots, k$, posets describing individual resource dimensions (e.g., cost, power, energy capacity). The *robot resource space* is the product poset $\mathcal{C} := \prod_{i=1}^k \mathcal{C}_i$, equipped with the standard product order.

2) *Planner Modeling*: Since the planner’s objective is to generate, for each robot R_i in the fleet $\mathcal{F}$, a sequence of

waypoints such that the fleet accomplishes the given task on the map M , we first define the notion of a robot’s waypoints.

Definition 20 (Waypoints of a robot). The *waypoint sequence* assigned to robot R_i is an ordered list

$$W_i = (w_{i,1}, \dots, w_{i,p_i}), \quad w_{i,j} \in M,$$

where $p_i \in \mathbb{N}$ may depend on the robot and the planning instance.

Definition 21 (Collection of waypoints). The *collection of waypoints* for a fleet $\mathcal{F} = (R_1, \dots, R_n)$ is the tuple $\mathbf{W} = (W_1, \dots, W_n)$, where W_i is the waypoint sequence assigned to robot R_i .

C. Planning for multiple maps

In many applications, planners and fleet designs must generalize across multiple environments rather than a single fixed map. We capture this by introducing a finite set of maps.

Definition 22 (Set of maps). A *set of maps* is a finite set $\mathcal{M} = \{M_1, \dots, M_p\}$, where each M_i is a map in the sense of Definition 7.

Applying a planner to each M_i yields a corresponding set of waypoint collections.

Definition 23 (Set of waypoint collections). Given a fleet $\mathcal{F}$ and planner planner , the *set of waypoint collections* induced by $\mathcal{M}$ is

$$\mathcal{W} = \{\mathbf{W}_1, \dots, \mathbf{W}_p\}, \quad \mathbf{W}_i := \text{planner}(\mathcal{F}, M_i).$$

Similarly, applying the executor to each waypoint collection $\mathbf{W}_i$ yields a corresponding set of trajectory collections.

Definition 24. Given a fleet $\mathcal{F}$ and a time budget vector τ for the maximum allowed operating duration for each robot, the *set of trajectory collections* induced by the *set of waypoint collections* is

$$\mathcal{T} = \{\mathbf{J}_1, \dots, \mathbf{J}_p\}, \quad \mathbf{J}_i := \text{executor}(\mathcal{F}, \mathbf{W}_i, \tau).$$

Definition 25 (Evaluation over multiple maps). For a set of maps $\mathcal{M} = \{M_1, \dots, M_p\}$ and the corresponding set of trajectory collections $\mathcal{T} = \{\mathbf{J}_1, \dots, \mathbf{J}_p\}$ induced by a fixed fleet, planner, and time budget vector, the definition of a performance metric remains unchanged: each metric is still evaluated on a single pair $(\mathbf{J}_i, M_i)$.

To evaluate performance over multiple environments, we therefore consider a family of performance metric instances applied to selected map–trajectory pairs. Concretely, if

$$\mathcal{I} \subseteq \{1, \dots, q\} \times \{1, \dots, p\}$$

is an index set specifying which metric is evaluated on which map, then the associated multi-map evaluator is

$$\text{evaluator}_{\mathcal{M}}(\mathcal{F}, \mathcal{T}, \mathcal{M}) := (m_\ell(\mathcal{F}, \mathbf{J}_i, M_i))_{(\ell,i) \in \mathcal{I}}.$$

This allows multiple metrics to be evaluated on the same map–trajectory pair, e.g., coverage and number of detected targets on M_i .

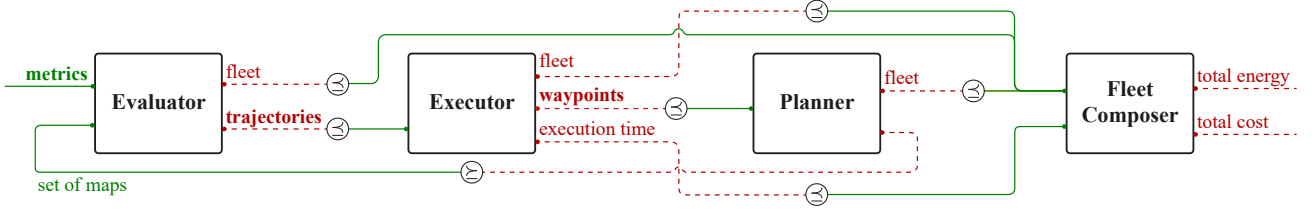


Fig. 2. Overview of our general co-design model. Bold wires indicate higher-order collections (i.e., sets of sets).

Remark 5. This construction enables evaluating a fixed fleet (and fixed planner) across multiple environments. More generally, one may compare different planners by evaluating their induced waypoint-collection sets over the same $\mathcal{M}$, while keeping the fleet design fixed.

V. FORMAL CO-DESIGN MODEL

In this section, we build on the formalization introduced in Section IV and cast the heterogeneous multi-robot system design problem as an optimization problem. Concretely, we seek to jointly determine (i) the design of the individual robots, (ii) the fleet composition, and (iii) the planning procedure so as to satisfy a desired task profile while trading off global resources such as cost and energy. To do so in a principled and compositional way, we leverage the monotone theory of co-design [6].

This section is structured as follows. Section V-A summarizes the monotone co-design formalism. Then, Sections V-B to V-E model the fleet composer, planner, executor, and evaluator as monotone design problems with implementation (MDPIs). Finally, Section V-F interconnects these MDPIs into the global co-design problem shown in Figure 2.

A. Background on the Monotone Theory of Co-Design

a) *Formulating Co-Design Problems:* The core modeling primitive of monotone co-design is the *monotone design problem with implementation* (MDPI).

Definition 26 (MDPI). Let posets $\mathcal{F}$ and $\mathcal{R}$ denote, respectively, the spaces of **functionalities** and **resources**. A MDPI is a triple $d := (\mathcal{I}_d, \text{prov}, \text{req})$, where $\mathcal{I}_d$ is a set of implementations and $\text{prov} : \mathcal{I}_d \rightarrow \mathcal{F}$, $\text{req} : \mathcal{I}_d \rightarrow \mathcal{R}$ assign to each implementation the functionality it provides and the resources it requires. We depict this as

$$\mathcal{F} \xleftarrow{\text{prov}} \mathcal{I}_d \xrightarrow{\text{req}} \mathcal{R},$$

and compactly write $d : \mathcal{F} \rightarrow \mathcal{R}$. An implementation $i \in \mathcal{I}_d$ is *feasible* for a desired functionality $f \in \mathcal{F}$ under an available resource budget $r \in \mathcal{R}$ if

$$\text{prov}(i) \succeq_{\mathcal{F}} f \quad \text{and} \quad \text{req}(i) \preceq_{\mathcal{R}} r.$$

Equivalently, each MDPI induces the feasibility map

$$\bar{d} : \mathcal{F}^{\text{op}} \times \mathcal{R} \rightarrow (\text{Pow}(\mathcal{I}_d), \subseteq), \\ (f^*, r) \mapsto \{i \in \mathcal{I}_d \mid (\text{prov}(i) \succeq_{\mathcal{F}} f) \wedge (\text{req}(i) \preceq_{\mathcal{R}} r)\},$$

where $(\cdot)^{\text{op}}$ reverses the order of a poset.

In co-design diagrams, we depict an MDPI as a block with green wires on the left (**functionalities**) and red dashed wires on the right (**resources**), as illustrated in Figure 2.

Remark 6 (Monotonicity). The map $\bar{d}$ is monotone in both arguments. Indeed, if fewer **functionalities** are demanded (i.e., $f' \preceq f$), then every implementation that satisfies f also satisfies f' , and the feasible implementation set can only grow: $\bar{d}(f', r) \supseteq \bar{d}(f, r)$. Similarly, if more **resources** are available (i.e., $r' \succeq r$), then any implementation feasible under r remains feasible under r' , so $\bar{d}(f^*, r') \supseteq \bar{d}(f^*, r)$.

Remark 7 (Populating the models). The co-design framework is agnostic to how one specifies the implementation set $\mathcal{I}_d$. In practice, implementations can be populated via analytic relations (e.g., cost models), numerical computation (e.g., solving optimal control problems), or in a data-driven or on-demand fashion (e.g., via simulations or by solving instances of planning problems). For concrete examples in mobility and autonomy systems, see [6], [27]–[29].

A core tenet of the monotone theory of co-design is that MDPIs can be *composed* in series, in parallel, and in feedback (loop) to build larger systems (i.e., a multigraph of MDPIs). In series composition, the functionality provided by one MDPI is used as a resource required by another. In parallel composition, two MDPIs operate independently and their functionalities and resources are aggregated. In feedback (loop) composition, a functionality output is connected to a resource input to enforce internal consistency constraints. Interconnections are expressed through the posetal order: if a wire connects a provided functionality f to a required resource r , feasibility is enforced by the constraint $r \preceq f$. Intuitively, a downstream module cannot require more than the upstream module provides.¹ Crucially, the class of MDPIs is closed under these compositions, so the resulting interconnected multigraph is itself a valid MDPI [6].

b) *Solving Co-Design Problems:* Given a MDPI, one can query it in two dual ways. A **FixFunMinRes** query fixes desired **functionalities** and returns the Pareto-optimal (i.e., minimal) **resources** required to provide them. A **FixResMaxFun** query fixes available **resources** and returns the Pareto-optimal (i.e., maximal) **functionalities** achievable under that budget.

Definition 27. Given a MDPI d , one defines monotone maps

- $h_d : \mathcal{F} \rightarrow A\mathcal{R}$, mapping a functionality to the *minimum* antichain of resources providing it;

¹Formally, the co-design primitives endow the category of design problems with a traced monoidal structure (locally posetal), which guarantees well-posedness of feedback compositions; see [6] for details.

- $h'_d: \mathcal{R} \rightarrow \mathcal{AF}$, mapping a resource budget to the *maximum* antichain of functionalities provided by it.

Computing these maps for composite problems with feedback can be reduced to fixed-point computations. Under standard assumptions (complete posets and Scott-continuity), one can apply Kleene's fixed point theorem to obtain an algorithm that converges to the set of optimal solutions or certifies infeasibility [6], [8]. Importantly, this fixed-point-based approach avoids brute-force enumeration of all design combinations: its computational cost scales linearly with the number of implementations available in each constituent block (up to antichain operations), rather than combinatorially with the number of design decisions [6], [8].

B. The Fleet Design Problem

We now model the fleet design layer from Section IV within monotone co-design. At this layer, two decisions are tightly coupled: (i) selecting the design of each robot, and (ii) selecting how many robots are included in the fleet. We represent this with a composite MDPI called the **Fleet Composer**. It provides the **fleet** and associated **execution-time budgets** as functionalities, while requiring the **total resources** needed to realize and operate that fleet. Internally, this is constructed by interconnecting multiple **Robot** MDPIs, one per robot "slot" (see Figure 5).

1) *Robot MDPI*: We begin with the **Robot** MDPI shown in Figure 3. The underlying robot design problem selects hardware and low-level control components (e.g., battery, actuation, sensors). For system-level co-design, we expose only the quantities that couple robot design to planning, execution, and fleet-level resource trade-offs.

Assumption 2 (Robot reduction). For the purposes of system-level comparison and co-design, each robot can be uniquely characterized at the interface level by the tuple $(\text{cost}, \text{power}, \text{capacity}, \mathcal{D}, \mathcal{S})$. That is, all robot implementation details influence (i) planning and execution only through $\mathcal{D}$ and $\mathcal{S}$, and (ii) fleet-level resource trade-offs through acquisition cost, operational power, and available energy capacity.

Remark 8 (Choice of exposed robot properties). Recall that in Definition 19 resources are introduced abstractly. Assumption 2 does not restrict generality; it simply specifies the interface variables used in this paper. Other applications may expose additional properties (e.g., mass, payload, compute) or hide some of the above by absorbing them into other blocks.

Accordingly, the Robot MDPI provides as **functionalities** the dynamical properties (Definition 17), sensing capabilities (Definition 18), and available energy capacity (in Wh). It requires as **resources** the acquisition cost (USD) and operational power (W). To compare designs, we require partial orders on $\mathcal{D}$ and $\mathcal{S}$.

Definition 28 (Poset of dynamical properties and sensing capabilities). The **dynamical properties** and **sensing capabilities** are ordered according to the componentwise order, i.e.:

$$\mathcal{D}_1 \preceq \mathcal{D}_2 \iff d_{1,i} \preceq_{\mathcal{D}_i} d_{2,i} \quad \forall i = 1, \dots, w.,$$

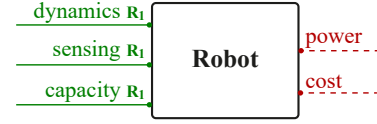


Fig. 3. The **Robot** MDPI.

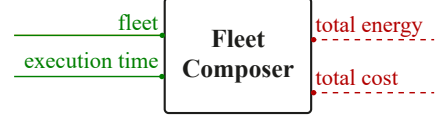


Fig. 4. The **Fleet Composer** MDPI.

$$\mathcal{S}_1 \preceq \mathcal{S}_2 \iff s_{1,j} \preceq_{\mathcal{S}_j} s_{2,j} \quad \forall j = 1, \dots, v.$$

Assumption 3 (Least elements). The posets of dynamical and sensing properties admit least elements $\perp_{\mathcal{D}}$ and $\perp_{\mathcal{S}}$, respectively, such that $\perp_{\mathcal{D}} \preceq \mathcal{D}$ and $\perp_{\mathcal{S}} \preceq \mathcal{S}$ for all $\mathcal{D}$ and $\mathcal{S}$.

Definition 29 (Null robot). A *null robot* $R_{\perp}$ is a robot that provides the least dynamical and sensing tuples $(\perp_{\mathcal{D}}, \perp_{\mathcal{S}})$, has zero energy capacity, and requires zero resources (zero cost and zero power). Its associated execution-time budget is $t^{\text{exec}}(R_{\perp}) = 0$.

Lemma 1 (Robot MDPI). The Robot MDPI is valid.

Proof. Consider two resource assignments $(\text{cost}_1, \text{power}_1)$ and $(\text{cost}_2, \text{power}_2)$ such that $\text{cost}_1 \leq \text{cost}_2$ and $\text{power}_1 \leq \text{power}_2$. Any robot design that is feasible under the smaller resource budget $(\text{cost}_1, \text{power}_1)$ remains feasible under the larger budget $(\text{cost}_2, \text{power}_2)$ by selecting the same component implementations and simply not utilizing the additional available resources. Consequently, increasing the available cost or power cannot reduce the achievable dynamical properties, sensing capabilities, or energy capacity of the robot. $\square$

2) *Fleet Composer MDPI*: We now turn to the **Fleet Composer** MDPI shown in Figure 4. It interconnects n Robot MDPIs to synthesize a fleet. The Fleet Composer provides as **functionalities** (i) the fleet and (ii) an execution-time budget for each robot. It requires as **resources** the total cost (USD) and total energy (Wh) associated with realizing and operating that fleet.

The key modeling question at this level is how to represent fleets in a way that supports comparison and is compatible with co-design composition. We expose to downstream planning and execution only *effective capabilities*, namely the dynamical and sensing tuples of each robot, together with an execution-time budget indicating how long these capabilities can be sustained.

a) *Fleet Representation and Order*: We represent a fleet as a finite collection of capability pairs

$$\mathcal{F} = ((\mathcal{D}_1, \mathcal{S}_1), \dots, (\mathcal{D}_n, \mathcal{S}_n)),$$

where each pair $(\mathcal{D}_i, \mathcal{S}_i)$ describes the effective capabilities of robot R_i as exposed to planning and execution. The corresponding execution-time budgets are collected in $\tau = (\tau_1, \dots, \tau_n) \in \mathbb{R}_{\geq 0}^n$. We order execution-time vectors componentwise using the standard order on $\mathbb{R}_{\geq 0}$.

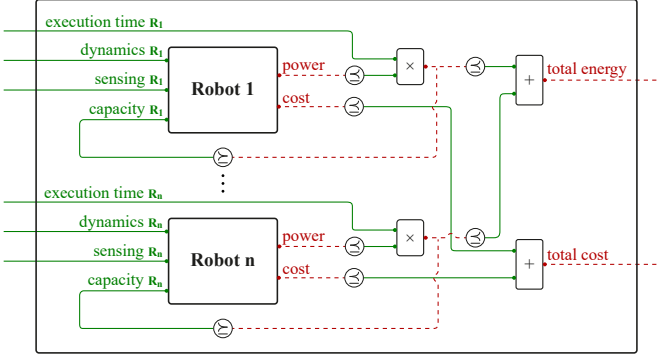


Fig. 5. Interconnection of multiple **Robot** MDPIs to form the **Fleet Composer** MDPI.

Definition 30 (Partial order on fleets). Let

$$\mathcal{F}_1 = ((\mathcal{D}_{1,1}, \mathcal{S}_{1,1}), \dots, (\mathcal{D}_{1,n}, \mathcal{S}_{1,n}))$$

and

$$\mathcal{F}_2 = ((\mathcal{D}_{2,1}, \mathcal{S}_{2,1}), \dots, (\mathcal{D}_{2,m}, \mathcal{S}_{2,m}))$$

be two fleets (possibly of different cardinalities). We write $\mathcal{F}_1 \preceq_F \mathcal{F}_2$ if and only if there exists an injective map

$$\varphi : \{1, \dots, n\} \hookrightarrow \{1, \dots, m\}$$

such that

$$\mathcal{D}_{1,i} \preceq \mathcal{D}_{2,\varphi(i)} \quad \text{and} \quad \mathcal{S}_{1,i} \preceq \mathcal{S}_{2,\varphi(i)} \quad \forall i \in \{1, \dots, n\}.$$

In words, every robot in $\mathcal{F}_1$ is dominated (in dynamical and sensing capabilities) by a *distinct* robot in $\mathcal{F}_2$.

Remark 9 (Unlabeled robots and antisymmetry). The indices in the tuple representation of $\mathcal{F}$ are bookkeeping labels. Since robots are interchangeable at the interface level (and may be permuted without changing system behavior), we identify fleets that differ only by permutation of robot indices. Under this identification, the relation $\preceq_F$ in Definition 30 is a partial order.

b) Fleet Composer Interconnection: The detailed Fleet Composer structure is shown in Figure 5. Each robot provides its power requirement and energy capacity. The Fleet Composer multiplies each robot's power by its execution-time budget to obtain the energy required to operate that robot for the allotted time. A feedback constraint enforces feasibility by requiring the provided capacity to dominate the required energy. Summing acquisition costs yields total cost; summing operating energies yields total energy.

This construction makes explicit the feedback between robot-level energy capacity choices and downstream execution requirements. Planning and execution determine how long each robot must operate, which fixes the required execution-time budgets and hence the required energy capacity at the robot level.

Lemma 2 (Fleet Composer MDPI). The Fleet Composer MDPI is valid.

Proof. The Fleet Composer is obtained by composing Robot MDPIs through series/parallel connections and a feedback



Fig. 6. The **Planner** MDPI.

loop. All intermediate arithmetic operations used in the interconnection (addition and multiplication on $\mathbb{R}_{\geq 0}$) are monotone maps. Therefore, by closure of MDPIs under these compositions [6], the resulting Fleet Composer is a valid MDPI and is monotone in its resource inputs. $\square$

Remark 10 (Fixed-length representation via null robots). In Figure 5, the Fleet Composer is depicted with a fixed number n of robot slots. This does not imply that every realized fleet contains n active robots. Unused slots can be filled with the null robot $R_{\perp}$ from Definition 29, which has zero cost and power, and provides zero capacity as well as $(\perp_{\mathcal{D}}, \perp_{\mathcal{S}})$ and $t^{\text{exec}}(R_{\perp}) = 0$. Such slots do not affect total resources and are interpreted as absent robots. Consequently, n specifies only the *maximum* fleet size supported by the diagram.

C. The Planner Design Problem

We next describe the **Planner** MDPI shown in Figure 6. This design problem abstracts the choice of planning software that, given a fleet and an environment, generates waypoint assignments (Definition 10).

Within co-design, the Planner MDPI **requires** a fleet and a set of maps and **provides** a set of waypoint collections. Intuitively, providing the planner with a more capable fleet or a more permissive environment cannot reduce the set of waypoint options available.

a) Waypoint Collections: To reason about planning outputs within co-design, we order waypoint collections by an injective subsequence relation that preserves waypoint order. This captures the idea that a planner that can generate “more” waypoint options (e.g., by covering larger regions or by offering additional waypoints that can be ignored downstream) provides a larger functionality.

Definition 31 (Poset of waypoint collections). Let

$$\mathbf{W}^{(1)} = \prod_{i=1}^n W_i^{(1)} \quad \text{and} \quad \mathbf{W}^{(2)} = \prod_{j=1}^m W_j^{(2)}$$

be two waypoint collections (with possibly different cardinalities), where each $W_i = (w_{i,1}, \dots, w_{i,p_i})$ is an ordered waypoint sequence. We define a partial order $\preceq_{\mathbf{W}}$ by declaring that $\mathbf{W}^{(1)} \preceq_{\mathbf{W}} \mathbf{W}^{(2)}$ if and only if there exists an injective map $\varphi : \{1, \dots, n\} \hookrightarrow \{1, \dots, m\}$ such that, for every $i \in \{1, \dots, n\}$, the sequence $W_i^{(1)}$ is an order-preserving subsequence of $W_{\varphi(i)}^{(2)}$, i.e., there exists a strictly increasing map

$$\sigma_i : \{1, \dots, p_i^{(1)}\} \rightarrow \{1, \dots, p_{\varphi(i)}^{(2)}\}$$

satisfying

$$w_{i,k}^{(1)} = w_{\varphi(i), \sigma_i(k)}^{(2)} \quad \forall k \in \{1, \dots, p_i^{(1)}\}.$$

The subsequence order in Definition 31 assumes that the executor abstraction is compatible with waypoint refinement: inserting additional intermediate waypoints into a waypoint sequence cannot invalidate the feasibility of executing the original sequence, since the executor may realize the same motion by traversing the matched subsequence and ignoring additional inserted waypoints. This is an abstraction choice used to preserve monotonicity of the planner–executor inter-connection.

b) *Sets of Maps*: Recall from Definition 22 that a set of maps is a finite set $\mathcal{M} = \{M_1, \dots, M_p\}$. We order sets of maps by injective inclusion.

Definition 32 (Poset of sets of maps). Let $\mathcal{M}^{(1)} = \{M_{1,1}, \dots, M_{1,n}\}$ and $\mathcal{M}^{(2)} = \{M_{2,1}, \dots, M_{2,m}\}$ be two sets of maps. We write $\mathcal{M}^{(1)} \preceq_{\mathcal{M}} \mathcal{M}^{(2)}$ if and only if there exists an injective map $\varphi : \{1, \dots, n\} \hookrightarrow \{1, \dots, m\}$ such that

$$M_{1,i} \subseteq M_{2,\varphi(i)} \quad \forall i \in \{1, \dots, n\}.$$

c) *Sets of Waypoint Collections*: For each map in a set of maps, the planner provides a corresponding waypoint collection.

Definition 33 (Poset of sets of waypoint collections). Let $\mathcal{W}^{(1)} = \{\mathbf{W}_{1,1}, \dots, \mathbf{W}_{1,n}\}$ and $\mathcal{W}^{(2)} = \{\mathbf{W}_{2,1}, \dots, \mathbf{W}_{2,m}\}$ be two sets of waypoint collections. We say $\mathcal{W}^{(1)} \preceq_{\mathcal{W}} \mathcal{W}^{(2)}$ if and only if there exists an injective map $\varphi : \{1, \dots, n\} \hookrightarrow \{1, \dots, m\}$ such that

$$\mathbf{W}_{1,i} \preceq_{\mathcal{W}} \mathbf{W}_{2,\varphi(i)} \quad \forall i \in \{1, \dots, n\},$$

where $\preceq_{\mathcal{W}}$ is the order from Definition 31.

Lemma 3 (Planner MDPI). The Planner MDPI is valid.

Proof. We consider the two resource dimensions separately.

Fleet monotonicity. Let $\mathcal{F}_1 \preceq_{\mathcal{F}} \mathcal{F}_2$ and fix a set of maps $\mathcal{M}$. By Definition 30, every robot in $\mathcal{F}_1$ is dominated by a distinct robot in $\mathcal{F}_2$. Any waypoint collection feasible for $\mathcal{F}_1$ on a given map remains feasible for $\mathcal{F}_2$ by assigning the same waypoint sequences to the corresponding dominating robots (and assigning arbitrary waypoint sequences to any additional robots). Thus, the set of waypoint collections is monotone in the fleet.

Map-set monotonicity. Let $\mathcal{M}^{(1)} \preceq_{\mathcal{M}} \mathcal{M}^{(2)}$. By Definition 32, for every $M_{1,i} \in \mathcal{M}^{(1)}$ there exists a distinct $M_{2,\varphi(i)} \in \mathcal{M}^{(2)}$ such that $M_{1,i} \subseteq M_{2,\varphi(i)}$. Any waypoint sequence whose elements lie in $M_{1,i}$ also lies in $M_{2,\varphi(i)}$, so waypoint collections valid for $\mathcal{M}^{(1)}$ remain valid for $\mathcal{M}^{(2)}$. Therefore, the output is monotone in the set of maps. $\square$

D. The Executor Design Problem

The **Executor** MDPI (Figure 7) instantiates the executor interface from Definition 13. It **requires** a fleet, a set of waypoint collections, and an execution-time budget vector, and **provides** a set of dynamically feasible trajectory collections.

To use trajectories within co-design, we equip them with a partial order that captures the idea that a trajectory can be safely extended in time (e.g., by waiting) and can be improved by stronger sensing.

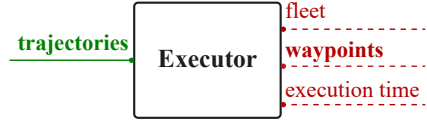


Fig. 7. The **Executor** MDPI.

Definition 34 (Poset of trajectories). Let $J_1 : [0, T_1] \rightarrow M$ and $J_2 : [0, T_2] \rightarrow M$ be two trajectories as in Definition 11. We associate with each trajectory a (time-invariant) sensing tuple $\mathcal{S}_{J_i} \in \mathcal{S}$ describing the sensing capabilities of the robot executing it. We define a partial order $\preceq_J$ by $J_1 \preceq_J J_2$ if and only if

- 1) **Prefix dominance in time and space**: $T_1 \leq T_2$ and $J_1(t) = J_2(t)$ for all $t \in [0, T_1]$;
- 2) **Sensing dominance**: $\mathcal{S}_{J_1} \preceq \mathcal{S}_{J_2}$, using the order from Definition 28.

Definition 35 (Poset of trajectory collections). Let $\mathbf{J}_1 = \{J_1^{(1)}, \dots, J_n^{(1)}\}$ and $\mathbf{J}_2 = \{J_1^{(2)}, \dots, J_m^{(2)}\}$ be two collections of trajectories. We write $\mathbf{J}_1 \preceq_{\mathbf{J}} \mathbf{J}_2$ if and only if there exists an injective map $\varphi : \{1, \dots, n\} \hookrightarrow \{1, \dots, m\}$ such that

$$J_i^{(1)} \preceq_J J_{\varphi(i)}^{(2)} \quad \forall i \in \{1, \dots, n\}.$$

Definition 36 (Poset of sets of trajectory collections). Let $\mathcal{T}^{(1)} = \{\mathbf{J}_{1,1}, \dots, \mathbf{J}_{1,n}\}$ and $\mathcal{T}^{(2)} = \{\mathbf{J}_{2,1}, \dots, \mathbf{J}_{2,m}\}$ be two sets of trajectory collections. We define $\mathcal{T}^{(1)} \preceq_{\mathcal{T}} \mathcal{T}^{(2)}$ if and only if there exists an injective map $\varphi : \{1, \dots, n\} \hookrightarrow \{1, \dots, m\}$ such that

$$\mathbf{J}_{1,i} \preceq_{\mathbf{J}} \mathbf{J}_{2,\varphi(i)} \quad \forall i \in \{1, \dots, n\},$$

where $\preceq_{\mathbf{J}}$ is from Definition 35.

Recall $\boldsymbol{\tau} = (\tau_1, \dots, \tau_n) \in \mathbb{R}_{\geq 0}^n$, the execution-time budget vector, ordered componentwise.

Lemma 4 (Executor MDPI). The Executor MDPI is valid.

Proof. We consider the three resource dimensions separately.

Fleet monotonicity. Let $\mathcal{F}_1 \preceq_{\mathcal{F}} \mathcal{F}_2$ and fix $(\mathcal{W}, \boldsymbol{\tau})$. Since every robot in $\mathcal{F}_1$ is dominated by a distinct robot in $\mathcal{F}_2$ (Definition 30), any dynamically feasible trajectory collection producible under $\mathcal{F}_1$ remains producible under $\mathcal{F}_2$ by assigning identical motions to the matched dominating robots and ignoring any additional capabilities. Thus, the set of trajectory collections is monotone in the fleet.

Waypoint monotonicity. Let $\mathcal{W}^{(1)} \preceq_{\mathcal{W}} \mathcal{W}^{(2)}$ and fix $(\mathcal{F}, \boldsymbol{\tau})$. For each waypoint collection $\mathbf{W}_{1,i} \in \mathcal{W}^{(1)}$ there exists a distinct $\mathbf{W}_{2,\varphi(i)} \in \mathcal{W}^{(2)}$ such that $\mathbf{W}_{1,i} \preceq_{\mathcal{W}} \mathbf{W}_{2,\varphi(i)}$. That is, by Definition 31, for each waypoint collection $\mathbf{W}_{1,i} \in \mathcal{W}^{(1)}$, there exists a distinct $\mathbf{W}_{2,\varphi(i)} \in \mathcal{W}^{(2)}$ such that each waypoint sequence in $\mathbf{W}_{1,i}$ is an order-preserving subsequence of the corresponding waypoint sequence in $\mathbf{W}_{2,\varphi(i)}$. Therefore, the Executor can realize the same trajectory collection by following the matched subsequence of waypoints while ignoring any additional inserted waypoints permitted by the larger sequence. Hence, any trajectory collection feasible for $\mathbf{W}_{1,i}$ remains feasible for $\mathbf{W}_{2,\varphi(i)}$, yielding $\mathcal{T}^{(1)} \preceq_{\mathcal{T}} \mathcal{T}^{(2)}$.

Time budget monotonicity. Let $\boldsymbol{\tau}^{(1)} \leq \boldsymbol{\tau}^{(2)}$ componentwise and fix $(\mathcal{F}, \mathcal{W})$. Any returned trajectory J_i under $\boldsymbol{\tau}^{(1)}$

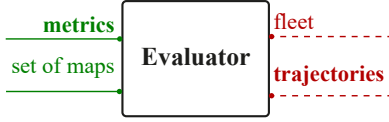


Fig. 8. The **Evaluator** MDPI.

satisfies $\text{dom}(J_i) = [0, T_i]$ with $T_i \leq \tau_i^{(1)}$ (Definition 13). The same trajectory is therefore feasible under $\tau^{(2)}$. If needed, the executor can extend trajectories by waiting after completion, which is consistent with the prefix order in Definition 34. Hence, the set of trajectory collections is monotone in τ . $\square$

Remark 11 (Interpretation of execution-time budgets). When the Executor MDPI provides a set of trajectory collections, the budget τ_i for robot R_i is interpreted as an upper bound on the execution horizon of *every* trajectory returned for that robot across the provided set. This convention is consistent with Definition 13 and with the trajectory order in Definition 34.

E. The Evaluator Design Problem

The **Evaluator** MDPI (Figure 8) assesses the performance of a fleet executing a task based on the trajectories produced by the Executor. It **requires** a fleet and a set of trajectory collections and **provides** (i) a set of maps on which the system can operate and (ii) a performance vector capturing task-relevant metrics (Definitions 14 and 15).

Assumption 4 (Least elements in performance spaces). Each performance space $(\mathcal{P}_i, \preceq_{\mathcal{P}_i})$ admits a least element $\perp_i$. This allows task profiles to selectively ignore metrics by setting their thresholds to $\perp_i$. For example, for coverage-percentage metrics, one can take $\perp_i = 0\%$.

Definition 37 (Poset of performance metric vectors). Let $\mathbf{P} := \mathcal{P}_1 \times \dots \times \mathcal{P}_q$ be the space of performance metric-value vectors. We equip $\mathbf{P}$ with the product order: for $\mathbf{m}_1 = (m_{1,1}, \dots, m_{1,q})$ and $\mathbf{m}_2 = (m_{2,1}, \dots, m_{2,q})$,

$$\mathbf{m}_1 \preceq_{\mathbf{P}} \mathbf{m}_2 \iff m_{1,i} \preceq_{\mathcal{P}_i} m_{2,i} \quad \forall i \in \{1, \dots, q\}.$$

Putting all of the above together, we are able to prove the monotonicity of the evaluator.

Lemma 5 (Evaluator MDPI). Assume each performance metric m_i is monotone in the fleet and in the trajectory collection (Definition 14). Then the Evaluator MDPI is valid.

Proof. We consider the two resource dimensions.

Trajectory-set monotonicity. Let $\mathcal{T}^{(1)} \preceq_{\mathcal{T}} \mathcal{T}^{(2)}$. By Definition 36, for every $\mathbf{J}_{1,j} \in \mathcal{T}^{(1)}$ there exists a distinct dominating $\mathbf{J}_{2,\varphi(j)} \in \mathcal{T}^{(2)}$ with $\mathbf{J}_{1,j} \preceq_{\mathbf{J}} \mathbf{J}_{2,\varphi(j)}$. Since each performance metric is monotone, the corresponding performance metric-value vector achieved on $\mathbf{J}_{2,\varphi(j)}$ dominates that of $\mathbf{J}_{1,j}$. Moreover, any map certified as feasible under $\mathcal{T}^{(1)}$ remains feasible under $\mathcal{T}^{(2)}$ because the evaluator may restrict attention to the dominating subset.

Fleet monotonicity. Let $\mathcal{F}_1 \preceq_{\mathcal{F}} \mathcal{F}_2$ and fix $\mathcal{T}$. By performance metric monotonicity in the fleet, evaluating the same trajectory collections under $\mathcal{F}_2$ yields metric values that

dominate those under $\mathcal{F}_1$. Similarly, any map certified feasible under $\mathcal{F}_1$ remains feasible under $\mathcal{F}_2$. $\square$

Remark 12 (Internal handling of auxiliary map information). Auxiliary map information is not exposed as an explicit wire in the co-design diagram. Instead, it is treated as an internal parameter of the performance metric definitions. This design choice keeps the evaluator interface minimal while allowing individual metrics to depend on additional task- or environment-specific information as needed. An example of auxiliary information is the locations of targets to be detected, which are not known a priori to the planner or executor.

F. Putting it All Together: The Global Co-Design Problem

Figure 2 illustrates the complete co-design optimization problem obtained by interconnecting the individual MDPIs introduced in Section V-B, Section V-C, Section V-D, and Section V-E.

At the highest level, the **Fleet Composer** MDPI requires the global resources **total cost** and **total energy**. In return, it provides a **fleet**, represented by effective dynamical and sensing capabilities, together with a vector of **execution-time budgets** specifying how long each robot can sustain these capabilities.

The **Planner** MDPI requires the **fleet** provided by the Fleet Composer together with a **set of maps** (the environments in which the system must operate). It produces a **set of waypoint collections** describing feasible waypoint assignments for the fleet across the considered maps.

The **Executor** MDPI requires this **set of waypoint collections**, along with the **fleet** and the corresponding **execution-time budgets**, and provides a **set of trajectory collections** that are dynamically feasible and respect the time budgets.

Finally, the **Evaluator** MDPI requires the resulting **set of trajectory collections** (and the **fleet**) and provides two global functionalities: (1) a **set of maps** that the system is certified to operate on, and (2) **performance metrics** quantifying the system's performance on those maps. These metrics constitute the ultimate functionality of the design problem and encode task requirements via the task profile (Definition 16).

Lemma 6 (Well-posedness of the global co-design problem). The interconnection in Figure 2 defines a composite MDPI.

Proof. Each block is modeled as a MDPI and is monotone with respect to its resource inputs (Lemmas 1 to 5). By the closure properties of monotone co-design under series, parallel, and feedback composition [6], the resulting interconnected multigraph is again a MDPI. $\square$

From a system-level perspective, the entire pipeline can therefore be interpreted as a single composite MDPI that **requires total cost** and **total energy** and **provides** the achieved task-level performance metrics. By querying this composite MDPI, one can directly identify Pareto-optimal heterogeneous multi-robot systems. For instance, a **FixFunMinRes** query can be used to determine all system designs that fulfill a specified task profile while minimizing the global resources (cost and energy). Importantly, the resulting Pareto front

not only quantifies trade-offs at the system level; it also specifies concrete implementation choices for each constituent block, including fleet composition, robot designs, and planner/executor selections.

Because the problem is compositional, each individual MDPI also represents a meaningful design problem on its own and can be queried independently. For example, the Fleet Composer can be queried in isolation to explore trade-offs in fleet design: `FixFunMinRes` identifies Pareto-optimal fleets that achieve specified dynamical and sensing properties while minimizing total cost and energy, whereas `FixResMaxFun` identifies the best achievable fleet capabilities under given cost and energy budgets. Analogous subsystem-level queries can be posed for robot design, planning, or execution, enabling targeted analysis without requiring evaluation of the entire pipeline.

Beyond formal correctness, this formulation highlights the central conceptual advantage of the proposed framework. The interfaces between modules are intentionally agnostic to implementation details: planners, executors, controllers, and evaluators can be replaced or extended without modifying the surrounding structure, as long as they respect the defined poset-valued interfaces. Similarly, tasks are specified abstractly via performance metrics and task profiles, which allows the framework to accommodate a wide range of multi-agent missions without structural changes.

Most importantly, the co-design formulation makes explicit the feedback between design layers that is inherent to heterogeneous multi-robot system design. Robot-level decisions (e.g., battery sizing, sensor suites, actuation choices) influence dynamical properties and energy availability, which in turn affect planning and execution feasibility. Conversely, the planner and executor determine execution-time requirements and trajectory feasibility, which feed back into robot and fleet design through the energy-capacity constraints. Likewise, the evaluator determines which maps and performance levels are achievable and feeds this information back to the planner. Capturing and exploiting these interdependencies in a principled and computationally tractable way is precisely what the monotone co-design framework enables.

G. Computational Advantage of Compositional Co-Design

A central motivation for the proposed formulation is computational. Designing a heterogeneous multi-robot system requires simultaneous choices over robot hardware, fleet composition, planning, execution, and task-level evaluation. If treated monolithically, these choices induce a very large Cartesian product of candidate system designs, and evaluating each candidate requires running the full pipeline. The compositional co-design formulation does not remove the need to model or simulate components, but it changes *how* this computation is organized: instead of enumerating all system-level combinations, it reasons locally through monotone interfaces and propagates only Pareto-relevant information across components.

We first quantify the size of the monolithic design space and then explain why the co-design formulation is computationally advantageous.

1) *Size of the Monolithic Design Space:* Suppose the design of each robot i is specified by n_i modules, and each module j admits $m_{i,j}$ candidate implementations. The number of designs for robot i is then

$$C_i = \prod_{j=1}^{n_i} m_{i,j}, \quad (2)$$

resulting in $\prod_i^N C_i$ design options for a heterogeneous fleet with N agents. Restricting the heterogeneity level, namely the number of robot types, makes the design space much smaller. Let T denote the number of robot types, $N_{\max}$ the maximum number of robots allowed per type. For a fixed type i , one may either deploy no robot of that type, or deploy $k \in \{1, \dots, N_{\max}\}$ robots of that type together with one of the C_i robot designs. Hence, the number of possible sub-fleet choices for type i is

$$1 + N_{\max} C_i. \quad (3)$$

Assuming independence across robot types, the total number of heterogeneous fleet designs is therefore

$$\prod_{i=1}^T (1 + N_{\max} C_i). \quad (4)$$

Suppose P is the number of planner implementations, and E is the number of executor implementations, then including those choices yields the total number of system-level design candidates

$$\begin{aligned} N_{\text{mono}} &= PE \prod_{i=1}^T (1 + N_{\max} C_i) \\ &= PE \prod_{i=1}^T \left(1 + N_{\max} \prod_{j=1}^{n_i} m_{i,j} \right). \end{aligned} \quad (5)$$

Equation (5) makes explicit the combinatorial nature of the problem: the search space is multiplicative across robot types and across the hardware/software design options within each type. Moreover, evaluating a single candidate requires running the complete pipeline, planning, trajectory generation, and task evaluation, so the cost of a monolithic brute-force search scales as

$$\mathcal{O}(N_{\text{mono}} \cdot c_{\text{full}}), \quad (6)$$

where c_{full} denotes the cost of one full system-level evaluation.

Some module choices may naturally be continuous rather than discrete. In that case, a monolithic method must still resolve them either through discretization or through repeated derivative-free evaluations of the full pipeline. For the purpose of counting candidate designs, this simply replaces $m_{i,j}$ by the number of effective evaluations used to resolve that module. The key point is unchanged: each additional continuous design variable increases the number of full-pipeline evaluations required by a monolithic approach.

Concrete size in our case studies: In the search-and-coverage instantiation of Section VI, we consider $T = 3$ robot types (medium aerial, large aerial, and ground), with at most $N_{\max} = 3$ robots per type, $P = 3$ planners, and $E = 1$ executor. Each robot type is defined by three modules: actuation, sensing, and battery. The actuation and sensing modules admit 2 and 1 choices, respectively. For the battery, we consider 8 battery technologies, each parameterized by a capacity value.

To obtain a concrete estimate, we discretize the battery capacity over the interval $(0, C_{\max}]$ with resolution r_{cap} , yielding

$$N_{\text{bat}}(r_{\text{cap}}) = \left\lfloor \frac{C_{\max}}{r_{\text{cap}}} \right\rfloor \quad (7)$$

capacity choices per battery technology, and therefore

$$8 N_{\text{bat}}(r_{\text{cap}}) \quad (8)$$

battery implementations per robot type. With $C_{\max} = 400 \text{ Wh}$ and $r_{\text{cap}} = 2 \text{ Wh}$, we obtain $N_{\text{bat}} = 200$.

Accordingly, the number of robot designs per type is

$$C_i = 2 \cdot 1 \cdot 8 N_{\text{bat}} = 3200, \quad i = 1, 2, 3. \quad (9)$$

The total number of fleet designs is therefore

$$(1 + 3 \cdot 3200)^3 = 9601^3, \quad (10)$$

and including planner/executor choices gives

$$N_{\text{mono}} = 3 \cdot 9601^3 \approx 2.66 \times 10^{12} \quad (11)$$

candidate system-level designs.

Even in this deliberately modest setup, brute-force evaluation of all candidates is clearly impractical. Importantly, this count only reflects the number of candidate designs; it does not yet account for the computational cost of evaluating each candidate through planning, execution, and simulation-based metric computation.

2) *Why Co-Design is Computationally Advantageous:* The advantage of co-design is not that it makes planning or simulation free. Rather, it avoids paying the full Cartesian-product cost in (5). The compositional formulation exploits the fact that the global design problem is not an unstructured black box: robot design, fleet composition, planning, execution, and evaluation interact through specific monotone interfaces, as captured by Figures 2 and 5.

This structure yields two computational benefits.

1) *Early pruning of dominated local designs:* In a monolithic search, every candidate system design is treated as a separate object and must be evaluated as such. In contrast, co-design reasons locally at the level of components and retains only Pareto-relevant information at each interface. Dominated local implementations are discarded as soon as they are recognized as such and therefore never participate in downstream combinations. For example, once a robot design is dominated by another design in terms of provided capabilities and required resources, there is no need to combine it with every planner, fleet multiplicity, and task instance.

2) *Solving by composition rather than enumeration:* The global problem is represented as an interconnection of monotone design problems. Series, parallel, and feedback compositions are handled directly in the co-design formalism, and feedback loops are resolved through fixed-point algorithms [6]. As a consequence, solving a system-level query does not require explicit traversal of all elements in the Cartesian product (5); instead, it consists of repeated propagation and pruning of antichains through the interconnection graph.

Therefore, the computational burden is governed by: (i) the size of the local implementation spaces, (ii) the widths of the intermediate antichains propagated through the interfaces, and (iii) the number of fixed-point iterations required by the feedback loops, rather than by the full number of monolithic candidates. This is precisely the structural advantage of the co-design approach: multiplicative coupling in the naive search space is replaced by local reasoning plus controlled composition.

Population cost versus query cost: It is useful to distinguish between two phases. First, one must *populate* the component-level design problems, which may still require simulation or numerical computation (e.g., evaluating planners and executors on relevant instances). Second, one must *solve* system-level co-design queries. The proposed framework does not eliminate the first cost, but it prevents the second phase from degenerating into a brute-force search over all global combinations. This distinction is especially important when multiple queries are asked over the same component library, since the populated component models can be reused across tasks, thresholds, or map sets.

In summary, the computational advantage of co-design lies in replacing the monolithic enumeration of (5) with compositional reasoning over monotone interfaces. For heterogeneous multi-robot systems, where the naive design space is already enormous even in modest examples, this structural reduction is essential to make system-level design queries tractable.

Remark 13 (Intellectual tractability). The benefit of the co-design formulation is not only computational. A monolithic optimization over robot design, fleet composition, planning, execution, and task evaluation is also difficult to interpret and manipulate: all couplings are hidden inside one large black-box search problem, making it hard to understand which subsystem drives a given trade-off or infeasibility. By exposing explicit interfaces between modules, the co-design formulation makes these dependencies legible. Designers can reason locally about robot design, fleet design, planning, or execution, and can modify one part of the architecture, for instance, by changing a planner, adding a robot type, or introducing a new task metric, without reformulating the entire problem. This modularity is especially valuable in collaborative settings, where different experts contribute models for different subsystems. Accordingly, the framework improves not only computational tractability but also the intellectual tractability of heterogeneous multi-robot system design.

VI. CASE STUDY: SEARCH AND COVERAGE

In this section, we instantiate the co-design pipeline developed above on a realistic heterogeneous multi-robot design problem. We focus on *search-and-coverage* missions, which capture a broad class of applications including search-and-rescue, environmental monitoring, and industrial inspection. The goal of the case studies is twofold: (i) to demonstrate end-to-end use of the proposed framework on a concrete domain, and (ii) to illustrate how changing the design space (robot components and planners) and the task profile (maps and performance metrics) changes the set of Pareto-optimal system designs, without changing the overall co-design structure.

We consider workspaces $M \in \mathcal{M}$ that are rectangular two-dimensional regions to be covered (i.e., $k = 2$ in Definition 7). For each workspace M , we further assume a (finite) set of target locations $\mathbf{x}_M^i \in M$, $i = 1, \dots, N_M$, representing objects whose positions are unknown to the planner and must be detected (e.g., gas leak sources in industrial inspection). Accordingly, we define two performance metrics:

- $m_c \in ([0, 1], \geq)$: the fraction of the map area covered by the fleet (higher is better);
- $m_\delta \in (\mathbb{N}, \geq)$: the number of targets detected with probability at least a specified confidence level $\delta \in (0, 1)$ (higher is better).

We formalize how coverage and detection are scored in Section VI-B, after introducing the available robot types and component catalogs used to populate the design space.

A. Robot Design

1) *Exposed dynamical properties*: In our case studies, each robot's dynamical properties $\mathcal{D}$ (Definition 17) comprise:

- $v_{\max} \in (\mathbb{R}_{\geq 0}, \geq)$: maximum velocity;
- $a_{\max, \text{lat}} \in (\mathbb{R}_{\geq 0}, \geq)$: maximum lateral acceleration;
- $a_{\max, \text{lon}} \in (\mathbb{R}_{\geq 0}, \geq)$: maximum longitudinal acceleration;
- $\dot{\theta}_{\max} \in (\mathbb{R}_{\geq 0}, \geq)$: maximum turning rate;
- $r_{\text{turn}} \in (\mathbb{R}_{\geq 0}, \leq)$: minimum turning radius;
- $c_{\text{vel}} \in (\mathbb{R}_{\geq 0}, \leq)$: power-velocity coefficient;
- $c_{\text{acc}} \in (\mathbb{R}_{\geq 0}, \leq)$: power-acceleration coefficient;
- $p \in \{\text{RS}, \text{DD}\}$: path type.

Here, RS denotes Reeds-Shepp and DD denotes differential-drive; these two types are incomparable [31]. The chosen partial orders reflect natural preferences: larger bounds on speed/acceleration/turn rate correspond to more capable motion, smaller turning radii correspond to improved maneuverability, and smaller power coefficients correspond to improved energetic efficiency.

2) *Power and energy model*: Given a trajectory with velocity profile $v(t)$ and acceleration profile $a(t)$, the instantaneous power consumption of a robot at time t is modeled as

$$P(t) = P_{\text{idle}} + c_{\text{vel}} v(t)^2 + c_{\text{acc}} |a(t)|, \quad (12)$$

where P_{idle} captures platform/sensor idling power and the remaining terms capture motion-dependent power draw.

Remark 14 (Energy accounting in the co-design instantiation). Evaluating eq. (12) requires access to the executed trajectory (and hence to the executor output). In our instantiation of

TABLE I
AVAILABLE BATTERY MODULES.

Technology	ρ (Wh/kg)	α (Wh/USD)
LiPo	250	2.50
LCO	195	2.84
LMO	150	2.84
NiH ₂	45	10.5
NiMH	100	3.41
LFP	90	1.50
NiCad	30	0.50
SLA	30	7.0

TABLE II
AVAILABLE ACTUATION MODULES.

Robot	Variant	$v_{\max}$ (m/s)	$a_{\max, \text{lat}}$ (m/s ²)	$a_{\max, \text{lon}}$ (m/s ²)	r_{turn} (m)	$\dot{\theta}_{\max}$ (rad/s)	c_{vel} (W s ² /m ²)	c_{acc} (W s ² /m)	$m_{\max}$ kg	P_{idle} (W)	Cost (USD)	Mass kg
Aerial (M)	V1	10.0	1.5	2.0	1.0	1.0	4	7	5	50	500	1
Aerial (M)	V2	13.0	1.5	2.0	1.0	1.0	5	9	7	70	600	1.5
Aerial (L)	V1	20.0	2.0	4.0	3	1.5	10	20	15	250	4000	4.0
Aerial (L)	V2	16.0	1.5	3.0	3	1.5	8	14	10	200	3500	3.5
Ground	V1	5.0	1.5	1.0	1.0	1.0	1	3	100	150	1500	10
Ground	V2	7.0	1.5	1.0	1.0	1.0	3	5	100	200	1750	12

the co-design graph, motion-dependent energy is obtained by integrating the trajectory-dependent terms in Equation (12), along the simulated execution. This allows the Executor to expose the energy required by a given plan, while the Fleet Composer enforces feasibility by ensuring that the selected battery capacity dominates the required energy, analogously to the execution-time coupling in Section V-B.

3) *Exposed sensing capabilities*: The sensing capabilities of each robot $\mathcal{S}$ (Definition 18) are characterized by the sensing radius $r_{\text{sensing}} \in (\mathbb{R}_{\geq 0}, \geq)$ and the instantaneous detection rate, which is determined by $\lambda_{\text{base}} \in (\mathbb{R}_{\geq 0}, \geq)$, $\sigma_d \in (\mathbb{R}_{\geq 0}, \geq)$ and $\beta_v \in (\mathbb{R}_{\geq 0}, \leq)$ (described in Section VI-B).

4) *Component catalogs and robot types*: Rather than exploring the property spaces $\mathcal{D}$ and $\mathcal{S}$ directly, we consider a more realistic setup in which the system designer selects from a modular catalog of robot types, following the component-based MDPI model shown in Figure 9. This makes reasoning about component costs more faithful to practice, as trade-offs arise from discrete module choices rather than from assumed continuous spaces $\mathcal{D}$ and $\mathcal{S}$. Concretely, we consider three robot types: a medium-sized aerial robot, a large aerial robot, and a ground robot. Within each type, multiple design choices must be made: (i) the battery module (determining energy capacity), (ii) the sensing module (which determines the sensing radius and the instantaneous detection rate), and (iii) the actuation module (which determines the robot's dynamical properties). The available battery modules are shared across robot types. Each battery technology is characterized by its specific energy-to-mass ratio ρ (Wh/kg) and its energy-to-cost ratio α (Wh/USD). The available battery technologies are listed in Table I. Each robot type offers two actuation-module variants (summarized in Table II) and one sensing module (specified in Table III).

B. Coverage and Detection Models

A point $\mathbf{x} \in M$ is covered by a robot at position $\mathbf{r}$ if it lies within the robot's sensing disk, i.e., $\|\mathbf{x} - \mathbf{r}\| \leq r_{\text{sensing}}$. The coverage metric m_c is then the fraction of the workspace

TABLE III
AVAILABLE SENSING MODULES.

Robot	Variant	r_s (m)	λ_{base}	σ_d (m)	β_v (s/m)	P_{req} (W)	Cost (USD)	Mass (kg)
Aerial (M)	imp1	12	0.95	8	0.004	20	100	0.4
Aerial (L)	imp1	40	0.98	30	0.02	68	200	1.0
Ground	imp1	30	0.99	26	0.02	35	150	2.5

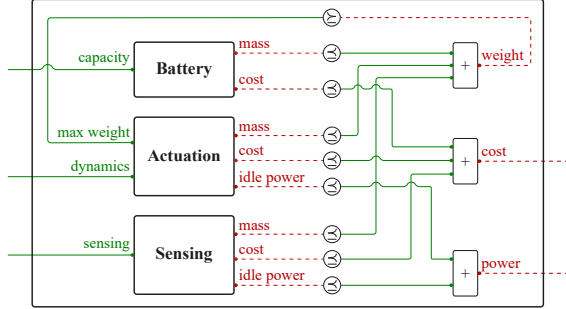


Fig. 9. The **Robot MDPI**.

covered by the union of sensing disks swept along all fleet trajectories.

Each robot's sensor is characterized by an instantaneous detection rate

$$\lambda(t, \mathbf{x}) = \lambda_{\text{base}} f(d(t, \mathbf{x})) g(\|v(t)\|), \quad (13)$$

where $d(t, \mathbf{x}) = \|\mathbf{r}(t) - \mathbf{x}\|$ is the distance between the robot position $\mathbf{r}(t)$ and a target location $\mathbf{x}$. The spatial decay and velocity-dependent degradation are modeled, respectively, as

$$f(d) = \exp\left(-\frac{d^2}{2\sigma_d^2}\right) \mathbf{1}\{d \leq r_{\text{sensing}}\}, \quad (14)$$

$$g(\|v\|) = \exp(-\beta_v \|v\|), \quad (15)$$

where σ_d controls the spatial uncertainty of the sensor and β_v captures the degradation of detection performance with increasing robot velocity. For a given target location $\mathbf{x}$, the cumulative exposure over a time horizon $[0, T]$ is

$$E(\mathbf{x}) = \int_0^T \lambda(t, \mathbf{x}) dt. \quad (16)$$

Modeling detection opportunities as independent micro-events and assuming that exposures add linearly across robots, the fleet-level probability of detecting a target at location $\mathbf{x}$ is

$$P_{\text{hit}}(\mathbf{x}) = 1 - \exp\left(-\sum_r E_r(\mathbf{x})\right), \quad (17)$$

where E_r is the cumulative exposure accumulated by robot r over its trajectory. The detection metric m_δ then counts the number of targets $\mathbf{x}_M^i$ for which $P_{\text{hit}}(\mathbf{x}_M^i) \geq \delta$.

C. Planning Algorithms

To populate the Planner MDPI with concrete implementations, we adapt three coverage planning algorithms from the literature: AGD [16], MRTA [18], and DARP [19]. All three planners output per-robot waypoint sequences, but differ substantially in how they decompose the workspace and allocate subregions to robots, which leads to qualitatively different waypoint structures (see Figure 10).

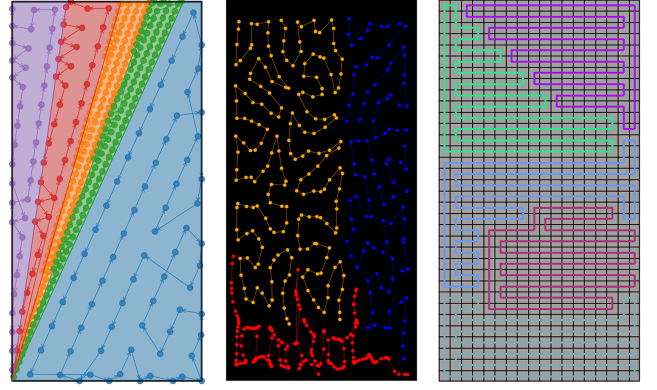


Fig. 10. Example waypoint solutions generated by three planners for a rectangular map of dimensions 400 m $\times$ 800 m. From left to right: AGD, MRTA, and DARP.

a) *AGD planner*: Given a robot with a known sensing radius and a designated subregion, the AGD algorithm [16] generates waypoints by adaptively tiling the area with grid cells fitted to the robot's sensing footprint, minimizing the number of waypoints needed to guarantee full coverage. To extend the method to heterogeneous multi-robot fleets, we partition the workspace into exclusive subregions using the anchored area decomposition of Hert and Lumelsky [32]. Each robot R_i receives a fraction of the total area proportional to $v_i r_{i,\text{sensing}}$, ensuring that faster robots with larger sensing footprints are assigned commensurately larger regions. The anchored area decomposition then partitions the workspace into exclusive subregions according to these weights, after which the AGD algorithm is applied independently to each robot-subregion pair. To determine the traversal order of the generated waypoints within each subregion, we apply the Christofides heuristic for the metric traveling salesman problem [33].

b) *MRTA planner*: Unlike AGD, the MRTA framework of Hu et al. [18] natively supports heterogeneous fleets. It decomposes the environment into subregions and allocates robot teams based on predicted coverage performance; each team then independently generates trajectories by minimizing an ergodic metric to achieve spatially uniform coverage. In our integration, we adopt only the subregion assignment and ergodic planning components, simplifying the original model by assuming uniform terrain across the workspace. Each robot's traversal dynamics are therefore taken directly from our actuation catalog (table II) rather than from the terrain-dependent velocity model of [18].

c) *DARP planner*: The DARP algorithm [19] computes an equitable, connected, and collision-free partition of a discretized workspace by iteratively assigning grid cells to robots until a balanced division is achieved. Since DARP is designed for homogeneous teams, we adapt it to our heterogeneous setting by setting the grid resolution to the sensing footprint of the smallest robot in the fleet, ensuring that every robot can cover any cell assigned to it. The standard DARP assignment procedure is then applied to obtain non-overlapping subregions, which serve as the basis for per-robot trajectory generation.

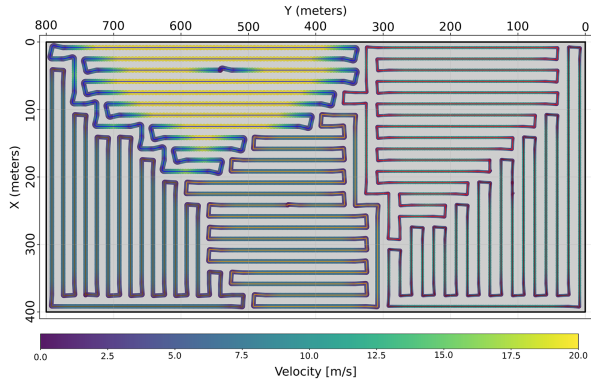


Fig. 11. Example trajectories generated by the **Executor** from a DARP waypoint assignment on a $400\text{ m} \times 800\text{ m}$ map. The resulting trajectories respect the motion constraints induced by the selected actuation modules.

Figure 10 shows representative waypoint solutions produced by the three planners on a $400\text{ m} \times 800\text{ m}$ workspace, illustrating the qualitative differences in spatial decomposition and waypoint structure across AGD, MRTA, and DARP.

D. Executor Details

The **Executor** maps a fleet, a waypoint assignment, and a time budget to dynamically feasible trajectories (Definition 13). In our case study, it is implemented through a simulation-based physics engine. Trajectory generation respects the motion constraints induced by the selected actuation module, including bounds on speed, acceleration, and turning rate as well as the path type (e.g., Reeds–Shepp versus differential drive). In the present case study, we use Reeds–Shepp trajectories for all robots, thereby fixing the trajectory model and isolating other design trade-offs. For each robot, the physics engine produces time-indexed state trajectories together with velocity and acceleration profiles, which are used to evaluate the power model in Equation (12) and compute the per-robot execution energy. The resulting trajectories and energy requirements are then passed to the **Evaluator** for task-level scoring and to the Fleet Composer for feasibility checks on battery sizing, respectively. Figure 11 shows an example trajectory generated for a DARP waypoint assignment on a $400\text{ m} \times 800\text{ m}$ map.

Remark 15 (Connection to the formal co-design model). The search and coverage task described above adheres to the interfaces of the co-design model formalized in Section V. In particular, all three planning algorithms conform to the fleet-map to waypoints interface regardless of their internal implementation, and the coverage and detection metrics follow the trajectory-based evaluation pipeline. For the Fleet Composer, since we consider three distinct robot types, we do not enumerate robots individually. Instead, the Fleet Composer contains one design block per type, together with a scalar wire representing the number of instances of that type. Aggregate quantities, such as total cost and energy, are obtained by scaling per-type resources and summing across types. This reduces the computational burden without altering the semantic interfaces.

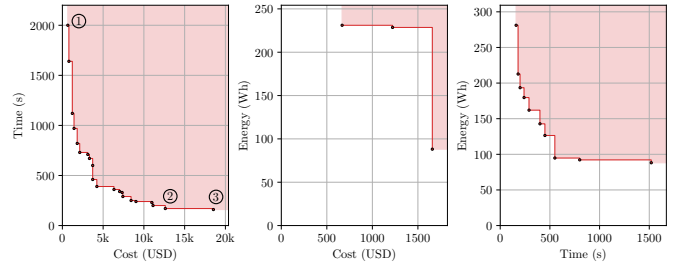


Fig. 12. Pairwise two-dimensional projections of the Pareto front for Case Study I. Left: **mission completion time** versus **total cost**. Middle: **total energy** versus **total cost**. Right: **total energy** versus **mission completion time**.

VII. NUMERICAL RESULTS

Here, we present four case studies that progressively stress different aspects of the framework. Case Study I demonstrates how a standard **FixFunMinRes** query exposes system-level trade-offs and returns the corresponding Pareto-optimal implementation choices. Case Study II compares our approach with sequential optimization baselines. Case Study III illustrates how additional structural constraints (here, a homogeneity constraint) can be enforced and optimized over. Finally, Case Study IV shows how the framework supports co-design under uncertainty by jointly optimizing across multiple potential deployment scenarios.

Throughout, we report trade-offs over three global resources: **total cost**, **total energy**, and **mission completion time** (makespan). We visualize Pareto-optimal designs through pairwise two-dimensional projections; each plotted point corresponds to a feasible multi-robot system that is nondominated in the relevant resource plane, together with a concrete selection of robot modules, fleet multiplicities, and planner.

A. Case Study I (Design Trade-offs)

We examine design trade-offs for a system tasked with covering a $400\text{ m} \times 800\text{ m}$ workspace. The fleet may include up to three robots per type (maximum fleet size of nine). We solve a **FixFunMinRes** query with required functionality fixed to $m_c \geq 0.95$ (i.e., at least 95% coverage), and the co-design solver minimizes the required system-level resources subject to this constraint.

Figure 12 shows three pairwise projections of the resulting Pareto front.

We highlight three points in the completion-time versus cost projection, which captures the speed–cost trade-off. Design ① minimizes cost: a single medium-sized aerial robot equipped with the lower-cost actuation module V1 and a 231.10 Wh NiMH battery, coordinated by DARP. This configuration completes the mission in 33 min 20 s at a cost of 667.77 USD. At the opposite extreme, design ③ minimizes completion time. It deploys two ground robots (14.97 Wh NiH₂ batteries), three medium-sized aerial robots (22.06 Wh NiH₂ batteries), and three large aerial robots (68.76 Wh NiH₂ batteries), with the higher-performance actuation module available (V2 for the ground and medium aerial robots, V1 for the large aerial robots). This fleet uses the AGD planner and finishes in 160 s, requiring 18,528.80 USD and 302.40 Wh.

Design ② illustrates the value of joint reasoning across design layers: three large aerial robots with 99.42 Wh batteries and V1 actuation, coordinated by DARP, complete the mission in 170 s at 12,628.40 USD, only 10 s slower than design ③ while saving 5,900.40 USD. Such a non-obvious trade-off would be difficult to identify without simultaneously optimizing robot modules, fleet multiplicities, and planner choice.

Overall, Case Study I shows that the framework produces interpretable Pareto fronts together with the implementation choices that realize each point, enabling principled exploration of system-level trade-offs.

B. Case Study II (Co-Design Vs. Sequential Optimization)

In the second case study, we compare the joint co-design approach with two sequential baselines that each fix one subsystem and optimize the other. The first baseline fixes the planner to AGD and optimizes only the robot designs and fleet composition. The second baseline fixes the robot designs—large aerial robot with V1 actuation, medium aerial robot, and ground robot with V2 actuation—and optimizes only the planner selection. Figure 13 compares the Pareto fronts obtained by these baselines against the full co-design solution from the previous case study.

As expected from the Pareto-optimality guarantees of co-design, the jointly optimized front lies on or below the sequential baselines across all three projections. Points shared between the co-design front and a sequential baseline correspond to designs that happen to remain optimal despite the fixed subsystem choice, whereas points appearing exclusively on the co-design front represent solutions that can only be uncovered when all design layers are optimized simultaneously. Notably, the sequential baseline fronts cross each other in the **time–cost** and **energy–time** projections, indicating that neither fixed-subsystem strategy uniformly dominates the other.

For a quantitative comparison, we report the hypervolume (HV), generational distance plus (GD+), and inverted generational distance plus (IGD+) indicators [34] for each front in Table IV; all fronts are normalized to $[0, 1]$ before computation. HV measures the volume between a dominated reference point and a given front and is maximized by the true Pareto front. GD+ and IGD+ are complementary distance-based indicators: GD+ measures the maximum distance from a candidate front to a reference front (capturing the worst-case gap), while IGD+ measures the maximum distance from the reference front to the candidate (capturing the worst-case omission). Both are zero if and only if the two fronts coincide, and lower values indicate closer agreement with the reference. We use the co-design Pareto front as the reference front for GD+ and IGD+, since it is guaranteed to dominate the sequential baselines by construction. For HV, we use the reference point $(1, 1, 1)$, corresponding to the worst normalized value in each objective. The results show that joint optimization improves the HV by up to 15% over the sequential baselines. Moreover, the degradation in HV and IGD+ is larger under the fixed-robots baseline, indicating that robot design choices have a greater impact on overall system performance than planner selection alone.

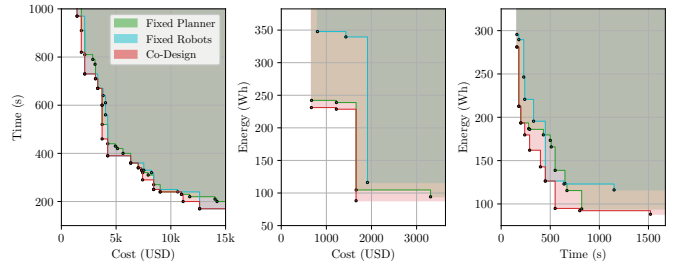


Fig. 13. Pairwise two-dimensional projections of the Pareto fronts for Case Study II, comparing co-design with sequential baselines. Left: **mission completion time** versus **total cost**. Middle: **total energy** versus **total cost**. Right: **total energy** versus **mission completion time**.

TABLE IV
QUALITY INDICATORS FOR THE CO-DESIGN AND SEQUENTIAL BASELINE PARETO FRONTS.

Metric	Co-Design	Fixed Planner	Fixed Robots
HV ($\uparrow$)	0.8471	0.8013	0.7358
GD+ ($\downarrow$)	0.0	0.0224	0.0203
IGD+ ($\downarrow$)	0.0	0.0139	0.0271

Finally, we report the computational effort required to obtain the results for this case study. The dominant cost lies in the simulation-based evaluation of planner and executor implementations (waypoint generation and trajectory execution), which requires approximately 16 hours of CPU time. The co-design solver itself adds only approximately 30 minutes, confirming that the framework introduces modest overhead relative to the underlying simulations.

C. Case Study III (Homogeneous vs. Heterogeneous)

This case study demonstrates how additional structural constraints on the fleet can be expressed and optimized over within the same pipeline. The task profile is identical to Case Study I ($m_c \geq 0.95$ coverage of a $400\text{ m} \times 800\text{ m}$ workspace), and we compare: (i) the unconstrained setting permitting heterogeneous fleets (as before), and (ii) a constrained setting restricting the Fleet Composer to ground robots only (maximum fleet size of nine).

Figure 14 reports two projections of the resulting Pareto fronts. In the homogeneous setting, all Pareto-optimal designs select DARP, consistent with its original design objective for homogeneous coverage planning. The dominance patterns differ across resource planes. In the cost–time projection, heterogeneous fleets strictly dominate: for every Pareto-optimal homogeneous design, there exists a heterogeneous alternative that is both faster and cheaper. In the energy–time projection, homogeneous ground fleets dominate the heterogeneous ones. This can be explained by platform characteristics: ground robots are priced between the medium and large aerial robots but have substantially lower maximum velocities (longer mission times), while their smaller power coefficients and lower operating speeds yield significantly reduced energy consumption (cf. Equation (12)).

Designs ① and ② illustrate the cost–time trade-off. Design ① is a homogeneous fleet of five ground robots with V2 actuation, coordinated by DARP, completing the mission

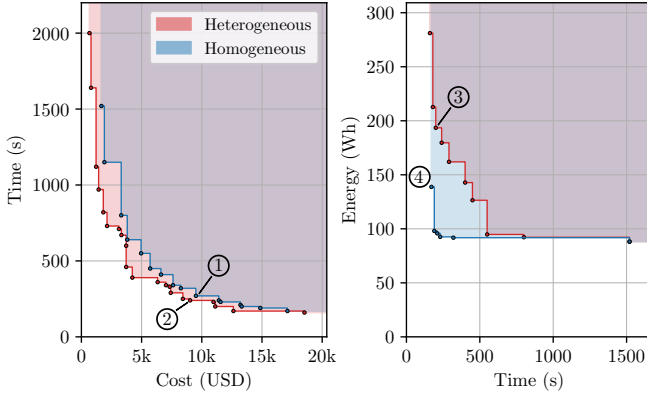


Fig. 14. Pairwise two-dimensional projections of the Pareto front for Case Study III. Left: **mission completion time** versus **total cost**. Right: **total energy** versus **mission completion time**.

in 270 s at 9,511.94 USD. Design ② is a heterogeneous fleet, one medium-sized and two large aerial robots under the AGD planner, that finishes in 240 s at only 9,029.13 USD. The heterogeneous fleet is thus simultaneously faster and cheaper. The energy–time projection tells a different story. Design ③, a heterogeneous fleet of three ground robots and three large aerial robots, completes the mission in approximately 200 s but consumes 193.50 Wh. Design ④, a homogeneous fleet of nine ground robots, finishes in 170 s while requiring only 138.80 Wh, demonstrating that homogeneous ground fleets can be both faster and more energy-efficient when aerial platforms are excluded.

This case study demonstrates that the co-design framework allows structural constraints on multi-agent systems (such as fleet homogeneity) to be specified systematically and optimized over, yielding Pareto-optimal designs together with the concrete implementation choices that realize them while adhering to additional design constraints.

D. Case Study IV (Task Uncertainty)

In this final case study, we demonstrate how the framework enables the design of multi-agent systems under uncertainty over deployment scenarios. We consider two workspaces: M_1 , a 400 m $\times$ 800 m region, and M_2 , a 500 m $\times$ 640 m region containing five targets at unknown locations, modeled using the detection framework introduced in Section VI-B. The system is optimized over three task profiles:

- **Task 1:** $m_c \geq 0.95$ on M_1 (coverage only).
- **Task 2:** Detect at least three targets on M_2 with per-target probability $\delta \geq 0.95$ (detection only).
- **Task 3:** Satisfy both Task 1 and Task 2 simultaneously.

Tasks 1 and 2 represent scenarios in which the deployment condition is known a priori and the fleet is optimized for a single mission. Task 3, by contrast, models uncertainty over the deployment scenario: the fleet must be capable of fulfilling either mission, requiring designs that are robust across both task profiles. We visualize the relation between the three tasks in Figure 15. The uncertain deployment setting is accommodated within our framework through the higher-order

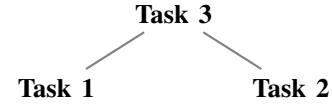


Fig. 15. Relation between the tasks in Case Study IV. A fleet that satisfies Task 3 is guaranteed to satisfy Task 1 and Task 2, while Task 1 and Task 2 are incomparable.

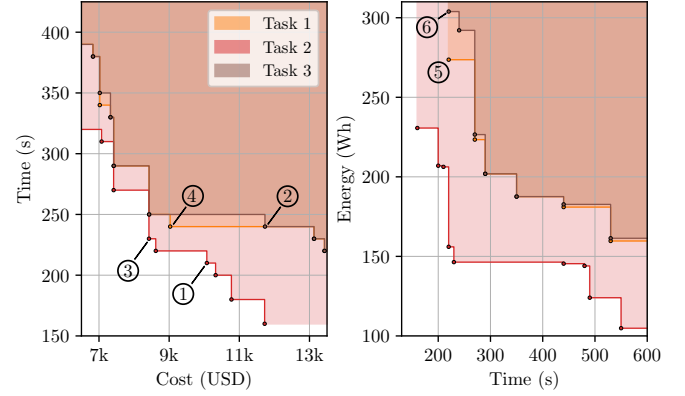


Fig. 16. Pairwise two-dimensional projections of the Pareto front for Case Study IV. Left: **mission completion time** versus **total cost**. Right: **total energy** versus **mission completion time**.

collections of waypoints, trajectories, and metrics introduced in Section IV-B.

Figure 16 shows the resulting Pareto fronts for Case Study IV. The figure consists of two pairwise projections: the left plot shows **mission completion time** versus **total cost**, while the right plot shows **total energy** versus **mission completion time**. Before analyzing individual implementations, we first examine the overall structure of the Pareto fronts. Empirically, the Pareto front for Task 1 is contained within that for Task 2 in both projections, indicating that (in this instantiation) achieving 95% coverage of M_1 is more resource-intensive than detecting three targets on M_2 at $\delta = 0.95$. Importantly, certain points on the Task 3 front coincide with neither the Task 1 nor the Task 2 front (e.g., design ②). These solutions are Pareto-optimal *only* under task uncertainty and therefore would not emerge from optimizing for either task in isolation.

Beyond these structural properties, changes in the task profile can fundamentally alter the optimal *planner* selection, even when budgets are comparable. Designs ③ and ④ illustrate this effect. For Task 1, a budget of \$9029.13 yields a heterogeneous fleet of one medium and two large aerial robots under the AGD planner (240 s). For Task 2, a comparable budget of \$8424.56 produces a different solution: two large aerial robots under the DARP planner (230 s). The shift to DARP is noteworthy: although traditionally associated with homogeneous coverage, its denser visitation patterns—induced by its use of the minimum sensing radius in heterogeneous fleet settings—increase cumulative exposure (cf. Equations (16) and (17)), making it advantageous for probabilistic detection even when it is suboptimal for strict coverage efficiency. We also observe instances where the *same* fleet composition

remains Pareto-optimal across Task 1 and Task 2 while the co-design solution switches the planner between AGD (coverage) and DARP (detection), highlighting that planner choice is itself a first-class design variable whose optimality can depend strongly on task structure.

Finally, designs ⑤ and ⑥ reveal a non-intuitive resource dependency. Both correspond to the same fleet (two robots of each type with identical actuation and battery configurations), yet they exhibit different energy requirements across tasks. Although Task 2 is generally less demanding when comparing Pareto-optimal designs, this particular configuration consumes more energy on Task 2 than on Task 1. Since the energy requirement under Task 3 is the maximum of the per-task energy demands, this yields different Pareto-optimal resource trade-offs under uncertainty. This illustrates that resource usage depends not only on task “difficulty” in isolation, but also on the interaction between fleet configuration, planner-induced visitation patterns, and task-specific metrics—dependencies that the co-design framework captures explicitly.

Overall, Case Study IV demonstrates that the framework naturally extends to multi-task settings and uncertain deployment conditions. By jointly optimizing robot design, fleet composition, and planning across tasks, it identifies Pareto-optimal designs that would not emerge from single-task optimization and exposes non-obvious interactions between planner structure, fleet heterogeneity, and task metrics.

VIII. CONCLUSION AND FUTURE WORK

In this paper, we introduced a formal, compositional, and task-aware framework to co-design the hardware and algorithms of heterogeneous multi-agent robotic systems. General, solution-agnostic abstractions of robots, fleets, planners, executors, and evaluators are provided, and domain experts are free to explore new designs that fit into them, without worrying about integration with other components. We then derive co-design models based on these abstractions, and interconnect them to globally optimize the entire system for various types of tasks. Modularity of co-design allows for more sophisticated models by further decomposing the components into sub-components. The compositional solver provided by co-design efficiently explores the combinatorial, large design space, without exhaustively simulating all combinations. Several case studies on coverage/search tasks demonstrate (i) how the co-design framework is applied to obtain system optimal design solutions for a specific task profile; (ii) the advantage of heterogeneous robot fleets compared to homogeneous ones; and (iii) how different task profiles affect the optimal design solutions.

As a first step in a general, compositional, collaborative, and interpretable task-aware co-design framework for heterogeneous robot fleets, this paper paves the way to universal *robotics phase diagrams*. There are many other aspects to include in the framework, with a few named here. We assume a centralized planner for waypoint generation and local controllers for tracking and execution. Incorporating communication capabilities as design variables and studying partially distributed algorithms and their trade-offs is an important

direction for future work. Although the abstractions and co-design problems are general, small collections of robot types, tasks, and planners are considered in this paper. Exploring larger catalogs will get us closer to the final robot phase diagram. Recent work [35] introduces distributional uncertainty and adaptive decision processes into co-design. With risk and uncertainty as important factors in real designs, incorporating them is a crucial step. Finally, some designs are expensive to evaluate (involve experiments, computationally heavy simulations, etc.), calling for a performance-aware online learning method to cleverly decide new solutions to explore.

REFERENCES

- [1] P. R. Wurman, R. D’Andrea, and M. Mountz, “Coordinating Hundreds of Cooperative, Autonomous Vehicles in Warehouses,” *AI magazine*, vol. 29, no. 1, pp. 9–9, 2008.
- [2] T. Standley, “Finding Optimal Solutions to Cooperative Pathfinding Problems,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 24, no. 1, 2010, pp. 173–178.
- [3] E. Galceran and M. Carreras, “A Survey on Coverage Path Planning for Robotics,” *Robotics and Autonomous Systems*, vol. 61, no. 12, pp. 1258–1276, 2013.
- [4] P.-S. Murvay and I. Silea, “A Survey on Gas Leak Detection and Localization Techniques,” *Journal of Loss Prevention in the Process Industries*, vol. 25, no. 6, pp. 966–973, 2012.
- [5] F. Z. Saberifar, D. A. Shell, and J. M. O’Kane, “Charting the Trade-off Between Design Complexity and Plan Execution Under Probabilistic Actions,” in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 135–141.
- [6] G. Zardini, “Co-design of Complex Systems: From Autonomy to Future Mobility Systems,” Ph.D. dissertation, ETH Zurich, 2023.
- [7] G. McFassel and D. A. Shell, “Reactivity and Statefulness: Action-based Sensors, Plans, and Necessary State,” *The International Journal of Robotics Research*, vol. 42, no. 6, pp. 385–411, 2023.
- [8] A. Censi, “A Mathematical Theory of Co-Design,” *arXiv preprint arXiv:1512.08055*, 2015.
- [9] L. Merino, F. Caballero, J. R. Martínez-de Dios, I. Maza, and A. Ollero, “An Unmanned Aircraft System for Automatic Forest Fire Monitoring and Measurement,” *Journal of Intelligent & Robotic Systems*, vol. 65, no. 1, pp. 533–548, 2012.
- [10] A. Mesbahi, F. Abbasi, and J. M. Velni, “A Team-based Deployment Approach for Heterogeneous Mobile Sensor Networks,” *Automatica*, vol. 106, pp. 327–338, 2019.
- [11] M. Rudolph, S. Wilson, and M. Egerstedt, “Range Limited Coverage Control using Air-Ground Multi-Robot Teams,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 3525–3530.
- [12] A. Breitenmoser, F. Tâche, G. Caprari, R. Siegwart, and R. Moser, “MagneBike: Toward Multi Climbing Robots for Power Plant Inspection,” in *Proceedings of the 9th International Conference on Autonomous Agents and Multiagent Systems: Industry track*, 2010, pp. 1713–1720.
- [13] W. DeBusk, “Unmanned Aerial Vehicle Systems for Disaster Relief: Tornado Alley,” in *AIAA Infotech@ Aerospace 2010*, 2010, p. 3506.
- [14] Y. Liu, M. Ficocelli, and G. Nejat, “A Supervisory Control Method for Multi-robot Task Allocation in Urban Search and Rescue,” in *2015 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*. IEEE, 2015, pp. 1–6.
- [15] J. P. Queralta, J. Taipalmaa, B. C. Pullinen, V. K. Sarker, T. N. Gia, H. Tenhunen, M. Gabbouj, J. Raitoharju, and T. Westerlund, “Collaborative Multi-Robot Search and Rescue: Planning, Coordination, Perception, and Active Vision,” *IEEE Access*, vol. 8, pp. 191 617–191 643, 2020.
- [16] S. Kazemdehbash, “Adaptive Grid-based Decomposition for UAV-based Coverage Path Planning in Maritime Search and Rescue,” *arXiv preprint arXiv:2412.00899*, 2024.
- [17] R. Almadhoun, T. Taha, L. Seneviratne, and Y. Zweiri, “A Survey on Multi-robot Coverage Path Planning for Model Reconstruction and Mapping,” *SN Applied Sciences*, vol. 1, no. 8, p. 847, 2019.
- [18] J. Hu, H. Coffin, J. Whitman, M. Travers, and H. Choset, “Large-Scale Heterogeneous Multi-robot Coverage via Domain Decomposition and Generative Allocation,” in *International Workshop on the Algorithmic Foundations of Robotics*. Springer, 2022, pp. 52–67.

- [19] A. C. Kapoutsis, S. A. Chatzichristofis, and E. B. Kosmatopoulos, "DARP: Divide Areas Algorithm for Optimal Multi-Robot Coverage Path Planning," *Journal of Intelligent & Robotic Systems*, vol. 86, no. 3, pp. 663–680, 2017.
- [20] I. Maza and A. Ollero, "Multiple UAV Cooperative Searching Operation Using Polygon Area Decomposition and Efficient Coverage Algorithms," in *Distributed Autonomous Robotic Systems 6*. Springer, 2007, pp. 221–230.
- [21] Y. Kantaros, M. Thanou, and A. Tzes, "Distributed coverage control for concave areas by a heterogeneous robot-swarm with visibility sensing constraints," *Automatica*, vol. 53, pp. 195–207, 2015.
- [22] H. Surmann, R. Worst, T. Buschmann, A. Leinweber, A. Schmitz, G. Senkowski, and N. Goddemeier, "Integration of UAVs in Urban Search and Rescue Missions," in *2019 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*. IEEE, 2019, pp. 203–209.
- [23] J. S. Lewis, D. A. Feshbach, and J. M. O’Kane, "Guaranteed Coverage with a Blind Unreliable Robot," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 7383–7390.
- [24] D. A. Shell, J. M. O’Kane, and F. Z. Saberifar, "On the Design of Minimal Robots That Can Solve Planning Problems," *IEEE Transactions on Automation Science and Engineering*, vol. 18, no. 3, pp. 876–887, 2021.
- [25] L. Carlone and C. Pinciroli, "Robot Co-design: Beyond the Monotone Case," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 3024–3030.
- [26] A. Censi, J. Lorand, and G. Zardini, "Applied Compositional Thinking for Engineering," 2024, work-in-progress book.
- [27] D. Milojevic, G. Zardini, M. Elser, A. Censi, and E. Frazzoli, "CODEI: Resource-Efficient Task-Driven Co-Design of Perception and Decision Making for Mobile Robots Applied to Autonomous Vehicles," *IEEE Transactions on Robotics*, vol. 41, pp. 2727–2748, 2025.
- [28] M.-P. Neumann, G. Zardini, A. Cerofolini, and C. H. Onder, "On the Co-Design of Components and Racing Strategies in Formula 1," in *2024 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2024, pp. 2876–2881.
- [29] G. Zardini, N. Lanzetti, A. Censi, E. Frazzoli, and M. Pavone, "Co-Design to Enable User-Friendly Tools to Assess the Impact of Future Mobility Solutions," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 2, pp. 827–844, 2022.
- [30] A. Wilhelm and J. Hughes, "SwarmCoDe: A Scalable Co-Design Framework for Heterogeneous Robot Swarms via Dynamic Speciation," *arXiv preprint arXiv:2603.26240*, 2026.
- [31] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.
- [32] S. Hert and V. Lumelsky, "Polygon Area Decomposition for Multiple-Robot Workspace Division," *International Journal of Computational Geometry & Applications*, vol. 8, no. 04, pp. 437–466, 1998.
- [33] N. Christofides, "Worst-Case Analysis of a New Heuristic for the Travelling Salesman Problem," in *Operations Research Forum*, vol. 3, no. 1. Springer, 2022, p. 20.
- [34] M. Li and X. Yao, "Quality Evaluation of Solution Sets in Multiobjective Optimisation: A Survey," *ACM Computing Surveys (CSUR)*, vol. 52, no. 2, pp. 1–38, 2019.
- [35] Y. Huang and G. Zardini, "Distributional Uncertainty and Adaptive Decision-Making in System Co-design," *arXiv preprint arXiv:2603.14047*, 2026.